Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

Лабораторная работы №3
«Механизмы многопоточности»
по дисциплине «Операционные системы»

Выполнил:
студент гр. 5151003/40001
Корпусов Д. Н.

Работу принял:
Ассистент преподавателя Гавва Г. Д.

Санкт-Петербург
2026

Содержание

Введение 3
1. Теоретическая часть.... 4-5
2. Практическая часть 7-31
2.1 Реализация многопоточного подсчёта разложений числа.... 6-10
2.2 Реализация многопоточной быстрой сортировки.... 10-15
2.3 Реализация многопоточной сортировки слиянием.... 16-20
2.4 Моделирование задачи обедающих философов 21-28
2.5 Поиск расстановок шахматных королей на доске.... 28-33
Вывод.... 34
Листинг А «expr.cpp».... 36-39
Листинг Б «qsort.cpp» 40-46
Листинг В «msort.cpp».... 47-53
Листинг Г «phil.cpp» 54-64
Листинг Д «chess.cpp» 65-71

ВВЕДЕНИЕ

В данной лабораторной работе рассматриваются механизмы многопоточности и синхронизации потоков в операционных системах Windows и Linux. Многопоточные программы позволяют разделять выполнение задачи между несколькими потоками, что может повысить эффективность работы программы и лучше использовать ресурсы процессора.

Основное внимание в работе уделяется созданию потоков, организации их взаимодействия и защите общих данных от одновременного некорректного доступа. Для этого используются различные средства синхронизации: мьютексы, семафоры, события, критические секции и условные переменные.

Цель работы – изучить принципы разработки многопоточных программ, изучить программный интерфейс операционных систем для организации многопоточности, а также получить практические навыки организации взаимодействия потоков в многопоточных программах.

В ходе выполнения работы необходимо реализовать несколько программ, использующих разные механизмы синхронизации в Windows и Linux, проверить корректность их работы и убедиться в отсутствии активного ожидания, взаимных блокировок и некорректного доступа к общим данным.

1. Теоретическая часть

Многопоточность – это способ организации выполнения программы, при котором внутри одного процесса создаётся несколько потоков. Поток представляет собой независимую последовательность выполнения команд. Все потоки одного процесса имеют общее адресное пространство и могут обращаться к общим данным процесса, но каждый поток имеет собственный стек и собственное состояние выполнения.

Использование потоков позволяет выполнять несколько частей программы параллельно или псевдопараллельно. Это особенно полезно в задачах, где вычисления можно разделить на независимые части. Однако при работе нескольких потоков с общими данными возникает необходимость синхронизации, так как одновременное изменение одних и тех же переменных может привести к ошибкам.

Для защиты общих данных используются объекты синхронизации. Они позволяют ограничить доступ к критическим участкам программы, организовать ожидание событий и координировать выполнение потоков.

Mutex – это объект синхронизации, который обеспечивает взаимное исключение. В один момент времени мьютекс может быть захвачен только одним потоком. Остальные потоки, пытающиеся захватить занятый мьютекс, переходят в состояние ожидания. Мьютексы применяются для защиты общих переменных, очередей задач и других разделяемых ресурсов.

Критическая секция выполняет похожую функцию и также используется для взаимного исключения. В Windows объект CRITICAL_SECTION применяется для синхронизации потоков внутри одного процесса. Поток входит в критическую секцию перед работой с общими данными и выходит из неё после завершения работы.

Семафор – это объект синхронизации со счётчиком. Если значение счётчика больше нуля, поток может продолжить выполнение, при этом значение семафора уменьшается. Если значение равно нулю, поток блокируется до тех пор, пока другой поток не увеличит счётчик. Семафоры

удобно использовать для ограничения числа потоков, одновременно работающих с ресурсом, или для организации очереди задач.

Событие в Windows используется для уведомления одного или нескольких потоков о наступлении некоторого условия. Поток может ожидать перехода события в сигнальное состояние. После установки события ожидающие потоки продолжают выполнение. События удобно применять, например, для одновременного запуска заранее созданных рабочих потоков.

Условная переменная в Linux применяется совместно с мьютексом и позволяет потоку ожидать выполнения некоторого условия. Если условие не выполнено, поток засыпает, временно освобождая мьютекс. Другой поток после изменения общего состояния может подать сигнал условной переменной и разбудить ожидающий поток. Такой механизм позволяет избежать активного ожидания.

Важной задачей при разработке многопоточных программ является предотвращение взаимных блокировок. Взаимная блокировка возникает, когда несколько потоков ожидают освобождения ресурсов друг другом и ни один из них не может продолжить выполнение. Также необходимо избегать активного ожидания, при котором поток постоянно проверяет условие в цикле и бесполезно расходует процессорное время.

В данной работе многопоточность применяется для распределения вычислительной нагрузки между рабочими потоками. Потоки создаются заранее, после чего получают задачи из общей очереди или ожидают разрешения от управляющего потока. Общие данные защищаются соответствующими объектами синхронизации в зависимости от операционной системы и условий задания.

2. Практическая часть

2.1 Реализация многопоточного подсчёта разложений числа

Алгоритм решения основан на рекурсивном переборе разложений числа. Чтобы не учитывать одинаковые разложения в разном порядке, слагаемые рассматриваются в невозрастающем порядке. Для этого в рекурсивную функцию передаётся не только оставшаяся сумма, но и максимальное допустимое значение следующего слагаемого. Если очередное выбранное слагаемое равно part, то следующий рекурсивный вызов выполняется для остатка суммы sum - part, а новое максимальное значение слагаемого становится равным part. Когда остаток суммы становится равен нулю, это означает, что найден один корректный вариант разложения.

Многопоточность организована через разбиение задачи по первому слагаемому. Для числа N возможные первые слагаемые имеют значения от N – 1 до 1, так как разложение N = N не допускается. Каждое такое значение рассматривается как отдельная задача. Рабочий поток получает очередное первое слагаемое, после чего рекурсивно подсчитывает все продолжения данного разложения. После завершения своих задач поток добавляет локально накопленный результат в общий счётчик.

Для синхронизации используются два мьютекса и одно событие. Первый мьютекс защищает выдачу очередного первого слагаемого, чтобы разные потоки не получили одну и ту же задачу. Второй мьютекс защищает общий счётчик ответа. Событие используется для одновременного запуска всех рабочих потоков после того, как они уже созданы и начат замер времени работы алгоритма.

Таблица 1 – Реализованные функции программы expr.cpp

Функция	Назначение
CountParts(int sum, int maxPart)	Рекурсивно подсчитывает количество разложений остатка sum, разрешая использовать слагаемые не больше maxPart.

Продолжение таблицы 1

Функция	Назначение
CountForFirst(int first)	Считает количество разложений числа N, которые начинаются с заданного первого слагаемого first.
GetNextFirst()	Выдаёт рабочему потоку следующее первое слагаемое для обработки. Доступ к общей переменной защищается мьютексом.
AddAnswer(unsigned long long value)	Безопасно добавляет частичный результат потока к общему ответу g_answer.
WorkerThread(void* param)	Функция рабочего потока. Поток ждёт стартового события, берёт задачи, считает разложения и добавляет свой результат в общий ответ.
main()	Считывает входные данные, создаёт объекты синхронизации и рабочие потоки, запускает вычисления, ожидает завершения потоков и записывает результаты в файлы.

На рисунке 1 представлена блок-схема работы функции рабочего потока, используемой в программе expr.cpp. Каждый рабочий поток выполняет одинаковый алгоритм: ожидает общего сигнала запуска, получает очередную задачу, выполняет вычисления для этой задачи и накапливает частичный результат. После завершения всех доступных задач поток добавляет свой результат в общий ответ.

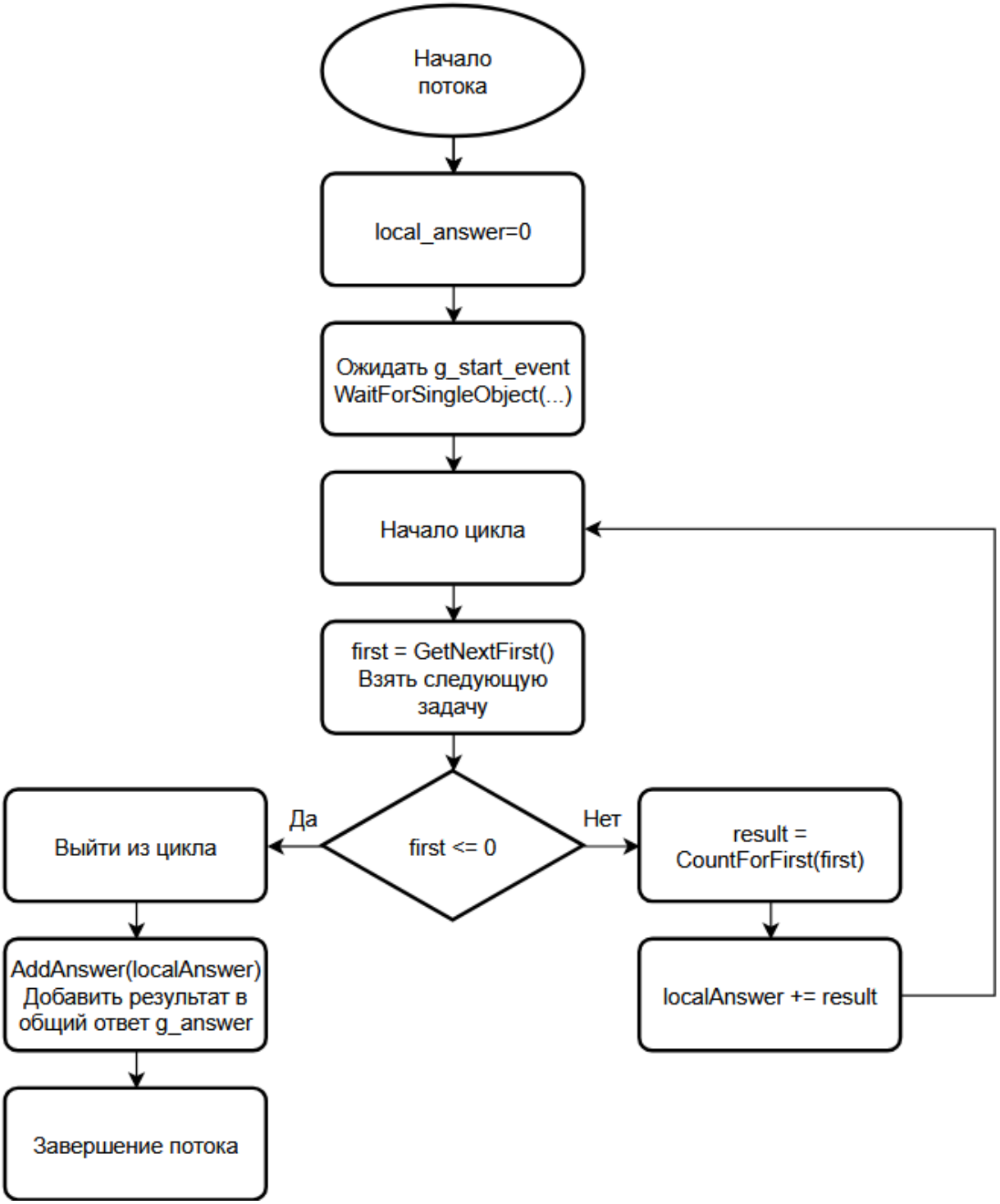


Рисунок 1 – Блок-схема функции рабочего потока WorkerThread программы expr.cpp

В начале работы поток инициализирует локальную переменную localAnswer, в которой хранится частичный результат. Затем поток ожидает событие g_start_event, чтобы начать вычисления одновременно с остальными потоками. После запуска поток в цикле получает очередное первое слагаемое с помощью функции GetNextFirst(). Если задач больше нет, поток выходит из цикла и добавляет накопленный результат в общий счётчик g_answer с

помощью функции AddAnswer(). Если задача получена, поток вызывает CountForFirst(first), прибавляет результат к localAnswer и возвращается к получению следующей задачи.

Для наглядного отображения взаимодействия потоков была составлена временная диаграмма. На ней показаны основные этапы работы главного потока и рабочих потоков, а также моменты, в которых потоки переходят в состояние ожидания из-за объектов синхронизации. Обозначение R соответствует состоянию ready/running, то есть поток готов к выполнению или выполняется. Обозначение B соответствует состоянию blocked, когда поток ожидает событие, мьютекс или завершение другого потока.

Время	-->	-->	-->	-->	-->	-->	-->
Поток	**1. Инициализация**	**2. Создание потоков**	**3. Ожидание старта**	**4. Получение задачи**	**5. Поиск решений**	**6. Запись ответа**	**7. Завершение**
Main	R читает input.txt, создает Event/Mutex	R CreateThread	R старт таймера, SetEvent	B WaitForSingleObject ожидает worker	B ожидает worker	B ожидает worker	R пишет output.txt, time.txt
Worker 1	—	R создан	B WaitForSingleObject (g_start_event)	R/B ждет g_task_mutex, берет first	R CountForFirst(first)	R/B ждет g_answer_mutex, g_answer += localAnswer	Завершен
Worker 2	—	R создан	B WaitForSingleObject (g_start_event)	R/B ждет g_task_mutex, берет first	R CountForFirst(first)	R/B ждет g_answer_mutex, g_answer += localAnswer	Завершен
Worker 3	—	R создан	B WaitForSingleObject (g_start_event)	R/B ждет g_task_mutex, берет first	R CountForFirst(first)	R/B ждет g_answer_mutex, g_answer += localAnswer	Завершен
...	—	R создан	B WaitForSingleObject (g_start_event)	R/B ждет g_task_mutex, берет first	R CountForFirst(first)	R/B ждет g_answer_mutex, g_answer += localAnswer	Завершен
Worker N	—	R создан	B WaitForSingleObject (g_start_event)	R/B ждет g_task_mutex, берет first	R CountForFirst(first)	R/B ждет g_answer_mutex, g_answer += localAnswer	Завершен

Рисунок 2 – Временная диаграмма взаимодействия потоков программы expr.cpp

Таблица 2 – Зависимость времени выполнения программы expr.cpp от числа потоков и значения N

№ теста	N	1 поток, мс	2	3	4	5	8	10	16
1	50	15	0	0	0	0	0	0	0
2	60	47	15	0	0	0	0	0	0
3	70	172	94	78	62	47	31	32	31
4	80	750	391	266	203	172	125	109	110
5	90	2859	1453	969	735	641	468	437	391
6	100	10140	5063	3438	2672	2140	1563	1391	1329
7	110	33688	16984	11343	8672	7172	5063	4563	4094
8	120	106750	53562	49765	27250	37687	17937	14125	12672

По результатам тестирования видно, что при увеличении значения N время выполнения программы значительно возрастает, так как увеличивается количество возможных разложений числа. Тестирование проводилось на процессоре *AMD Ryzen 7 7735H*, который имеет 8 физических ядер и 16 потоков выполнения. При увеличении количества рабочих потоков время выполнения в большинстве случаев уменьшается, особенно на больших значениях N.

Для малых значений N результаты менее показательны: время выполнения слишком мало, поэтому измерения могут давать значения 0 мс или не отражать реальную разницу между количеством потоков. В таких случаях накладные расходы на синхронизацию и планирование потоков сопоставимы с временем самих вычислений.

На больших значениях N многопоточность даёт заметное ускорение. Например, при N = 120 время выполнения уменьшается со 106750 мс на одном потоке до 12672 мс на 16 потоках. При этом ускорение не является строго пропорциональным количеству потоков, так как часть времени тратится на выдачу задач, синхронизацию доступа к общему ответу и переключение потоков. Наилучшие результаты в таблице достигаются при использовании 16 потоков, что соответствует количеству аппаратных потоков процессора.

Таким образом, многопоточная реализация эффективна при достаточно большой вычислительной нагрузке. Однако оптимальное количество потоков зависит от размера задачи и характеристик системы, а для малых задач выигрыш может быть незначительным или плохо измеримым.

2.2 Реализация многопоточной быстрой сортировки

Программа qsort.cpp реализует многопоточную быструю сортировку массива целых чисел по возрастанию. Входные данные считываются из файла input.txt: количество рабочих потоков, размер массива и элементы массива. После завершения сортировки программа записывает отсортированный массив в файл output.txt, а время выполнения алгоритма – в файл time.txt.

Основная идея алгоритма заключается в том, что сортируемый массив разбивается на отдельные подзадачи. Каждая подзадача представляет собой диапазон массива, который необходимо отсортировать. В начале работы в очередь задач помещается один диапазон – весь массив. Рабочие потоки берут задачи из общей очереди и обрабатывают их. Если диапазон достаточно большой, поток выполняет разбиение относительно опорного элемента, после чего левая и правая части добавляются в очередь как новые задачи. Если диапазон небольшой, он сортируется обычной рекурсивной быстрой сортировкой без дальнейшего добавления подзадач.

Для синхронизации потоков используются семафор и критическая секция. Семафор показывает наличие задач в очереди: если задач нет, рабочие потоки блокируются и не выполняют активное ожидание. Когда новая задача добавляется в очередь, семафор освобождается, и один из потоков может продолжить работу. Критическая секция защищает общую очередь задач и служебные счётчики от одновременного изменения несколькими потоками.

Завершение сортировки определяется с помощью счётчика незавершённых задач. В него входят как задачи, находящиеся в очереди, так и задачи, которые уже взяты потоками и выполняются. Когда этот счётчик становится равным нулю, это означает, что задач больше не осталось и сортировка завершена. После этого рабочие потоки пробуждаются, завершают выполнение, а главный поток записывает результаты в выходные файлы и освобождает ресурсы. Описаний функций qsort.cpp представлены в таблице 3.

Таблица 3 – Назначение функций, реализованных в файле qsort.cpp

Функция	**Назначение**
PushTask(int left, int right)	Добавляет новую задачу сортировки диапазона [left; right] в общую очередь задач и освобождает семафор, чтобы разбудить один рабочий поток.

Продолжение таблицы 3

Функция	**Назначение**
PushTask(int left, int right)	Добавляет новую задачу сортировки диапазона [left; right] в общую очередь задач и освобождает семафор, чтобы разбудить один рабочий поток.
PopTask(SortTask* task)	Ожидает появления задачи на семафоре и извлекает одну задачу из очереди. Возвращает 1, если задача получена, и 0, если работа завершена.
FinishTask()	Уменьшает счётчик незавершённых задач. Если задач больше не осталось, устанавливает флаг завершения и пробуждает все рабочие потоки.
Swap(int* a, int* b)	Меняет местами два элемента массива.
SimpleQuickSort(int left, int right)	Выполняет обычную рекурсивную быструю сортировку небольшого диапазона массива без создания новых подзадач.
ProcessTask(SortTask task)	Обрабатывает одну задачу сортировки: либо сортирует небольшой диапазон сразу, либо разбивает большой диапазон на две новые подзадачи.
WorkerThread(void* param)	Функция рабочего потока. Поток получает задачи из очереди, обрабатывает их и сообщает о завершении каждой задачи.
main()	Считывает входные данные, создаёт массив, очередь задач, объекты синхронизации и рабочие потоки, запускает сортировку, ожидает завершения потоков и записывает результат в файлы.

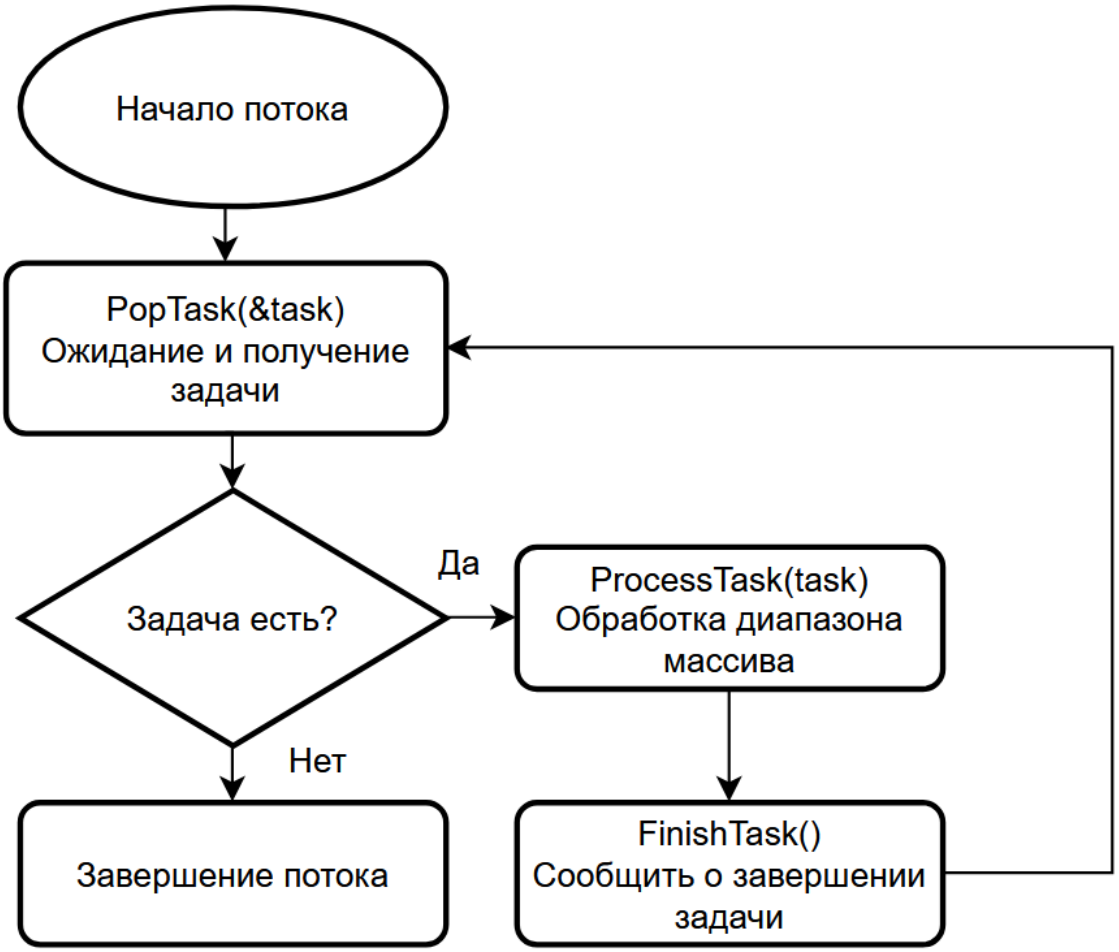


Рисунок 3 – Блок-схема функции рабочего потока WorkerThread программы qsort.cpp

На рисунке 3 представлена блок-схема алгоритма, выполняемого каждым рабочим потоком в программе многопоточной быстрой сортировки. Рабочий поток ожидает появления задачи в очереди, извлекает задачу, обрабатывает соответствующий диапазон массива и сообщает о завершении задачи. Если новых задач больше нет и установлен флаг завершения, поток выходит из цикла и завершает работу.

Рабочий поток начинает выполнение с вызова PopTask(). В этой функции поток блокируется на семафоре, если в очереди нет задач. После получения задачи поток вызывает ProcessTask(), где выполняется либо сортировка небольшого диапазона, либо разбиение большого диапазона на новые подзадачи. После обработки вызывается FinishTask(), уменьшающая счётчик незавершённых задач. Затем поток снова пытается получить новую

задачу. Если задач больше нет и сортировка завершена, PopTask() возвращает 0, после чего поток завершает выполнение.

На временной диаграмме, представленной на рисунке 4, показано взаимодействие главного потока и рабочих потоков при выполнении многопоточной быстрой сортировки. Диаграмма отражает основные моменты синхронизации: ожидание задач на семафоре, доступ к общей очереди задач через критическую секцию и ожидание главным потоком завершения рабочих потоков. Обозначение R соответствует состоянию ready/running, а B – состоянию blocked.

Время	--->	--->	--->	--->	--->	--->	--->
Поток	**1. Инициализация**	**2. Создание потоков**	**3. Ожидание задачи**	**4. Получение задачи**	**5. Обработка диапазона**	**6. Завершение задачи**	**7. Завершение**
Main	R читает input.txt, создает массив	R CreateThread	R старт таймера, PushTask(0, N-1)	B WaitForSingleObject ожидает worker	B ожидает worker	B ожидает worker	R пишет output.txt, time.txt
Worker 1	–	R создан	B WaitForSingleObject (g_task_sem)	R/B EnterCriticalSection, PopTask	R ProcessTask(task)	R/B EnterCriticalSection, FinishTask	Завершен
Worker 2	–	R создан	B WaitForSingleObject (g_task_sem)	R/B EnterCriticalSection, PopTask	R ProcessTask(task)	R/B EnterCriticalSection, FinishTask	Завершен
Worker 3	–	R создан	B WaitForSingleObject (g_task_sem)	R/B EnterCriticalSection, PopTask	R ProcessTask(task)	R/B EnterCriticalSection, FinishTask	Завершен
...	–	R создан	B WaitForSingleObject (g_task_sem)	R/B EnterCriticalSection, PopTask	R ProcessTask(task)	R/B EnterCriticalSection, FinishTask	Завершен
Worker N	–	R создан	B WaitForSingleObject (g_task_sem)	R/B EnterCriticalSection, PopTask	R ProcessTask(task)	R/B EnterCriticalSection, FinishTask	Завершен

Рисунок 4 – Временная диаграмма взаимодействия потоков программы qsort.cpp

Таблица 4 – Зависимость времени выполнения программы qsort.cpp от размера массива и количества потоков

№ теста	N	1 поток, мс	2	3	4	5	8	10	16
1	50000	0	0	0	0	0	0	0	0
2	100000	16	0	0	0	0	0	0	0
3	200000	31	15	15	15	15	15	0	0
4	5000000	922	484	328	266	234	188	156	125
5	7500000	1329	703	500	391	328	234	219	187
6	10000000	1735	891	625	516	438	313	297	234
7	15000000	2640	1375	1360	734	625	609	563	516
8	20000000	5093	2625	1813	1406	1187	812	719	672
9	25000000	5875	3218	2390	1953	1625	1172	1015	891
10	30000000	6843	4000	2812	2266	1875	1343	1188	891

Тестирование программы qsort.cpp проводилось на процессоре AMD Ryzen 7 7735H, который имеет 8 физических ядер и 16 потоков выполнения. По результатам тестирования видно, что при увеличении размера массива время сортировки возрастает, так как увеличивается количество элементов, которые необходимо обработать.

Использование нескольких потоков позволяет заметно сократить время выполнения сортировки. На малых размерах массива результаты менее показательны: время выполнения слишком мало, поэтому часть измерений получается равной 0 мс или близкой к этому значению. В таких случаях накладные расходы на создание задач, синхронизацию, работу семафора и критической секции могут быть сопоставимы с временем самой сортировки.

Для больших массивов многопоточность даёт более заметный эффект. Например, при размере массива 30 000 000 элементов время выполнения уменьшается с 6843 мс на одном потоке до 891 мс на 16 потоках. При этом ускорение не является строго линейным: увеличение числа потоков не всегда уменьшает время пропорционально, так как часть времени тратится на синхронизацию доступа к очереди задач, переключение потоков и работу планировщика ОС.

Наилучшие результаты в таблице в большинстве случаев достигаются при использовании 16 потоков, что соответствует количеству аппаратных потоков процессора. Таким образом, многопоточная реализация быстрой сортировки показывает эффективность на больших массивах, однако оптимальное число потоков зависит от размера входных данных и характеристик системы.

2.3 Реализация многопоточной сортировки слиянием

Программа msort.cpp реализует многопоточную сортировку слиянием массива целых чисел по возрастанию. Входные данные считываются из файла input.txt: количество рабочих потоков, размер массива и элементы массива. После завершения сортировки отсортированный массив записывается в файл output.txt, а время выполнения алгоритма – в файл time.txt.

Алгоритм реализован как модифицированная итерационная сортировка слиянием. Сначала исходный массив разбивается на небольшие блоки фиксированного размера. В данной реализации размер блока равен 1000 элементов. Каждый такой блок сортируется отдельно с помощью простой сортировки вставками. После этого программа уже работает не с отдельными элементами, а с готовыми отсортированными блоками.

Далее выполняются этапы слияния соседних отсортированных блоков. Сначала сливаются блоки размером 1000, затем 2000, затем 4000 и так далее, пока весь массив не станет одним отсортированным блоком. Каждая операция сортировки малого блока или слияния двух соседних блоков оформляется как отдельная задача.

Для выполнения задач используется пул рабочих потоков. Главный поток создаёт задачи текущего этапа и помещает их в общую очередь. Рабочие потоки ожидают появления задач, извлекают их из очереди и выполняют либо сортировку отдельного блока, либо слияние двух уже отсортированных частей массива. После завершения всех задач текущего этапа главный поток переходит к следующему этапу с увеличенным размером блока.

Синхронизация потоков реализована с помощью мьютекса и условных переменных библиотеки pthread. Мьютекс защищает общую очередь задач, счётчик задач текущего этапа и флаг завершения. Условная переменная g_cond используется для ожидания рабочими потоками появления новых задач, а условная переменная g_stage_cond - для ожидания главным потоком завершения текущего этапа сортировки.

Таблица 5 – Назначение функций, реализованных в файле msort.cpp

Функция	Назначение
PushTask(int left, int mid, int right)	Добавляет в очередь задачу слияния двух отсортированных диапазонов [left; mid] и [mid + 1; right], а затем подаёт сигнал рабочему потоку через условную переменную.
PopTask(SortTask* task)	Извлекает задачу из общей очереди. Если задач нет, рабочий поток блокируется на условной переменной g_cond до появления новой задачи или завершения программы.
FinishTask()	Уменьшает счётчик задач текущего этапа. Если все задачи этапа выполнены, будит главный поток через условную переменную g_stage_cond.
Merge(int left, int mid, int right)	Сливает два соседних отсортированных участка массива [left; mid] и [mid + 1; right] в один отсортированный участок.
ProcessTask(SortTask task)	Обрабатывает одну задачу, вызывая функцию Merge() для заданного диапазона массива.
WorkerThread(void* param)	Функция рабочего потока. Поток получает задачи из очереди, выполняет слияние и сообщает о завершении задачи текущего этапа.
ToMs(struct timespec* tm)	Переводит время из структуры timespec в миллисекунды. Используется для вычисления времени работы алгоритма.
WaitStage()	Блокирует главный поток до тех пор, пока все задачи текущего этапа сортировки слиянием не будут выполнены.
main()	Считывает входные данные, создаёт массивы, потоки и объекты синхронизации, организует этапы сортировки слиянием, ожидает завершения потоков и записывает результаты в файлы.

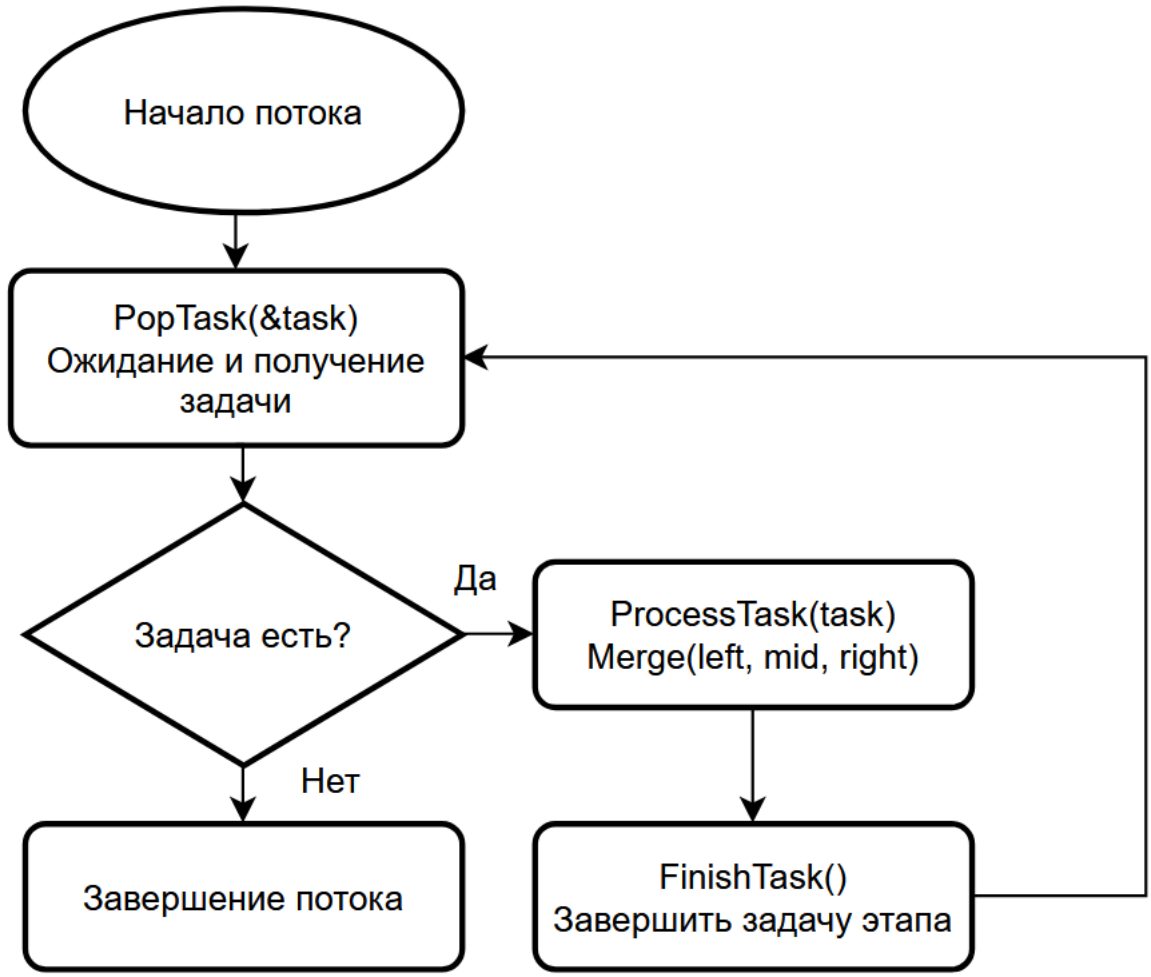


Рисунок 5 – Блок-схема функции рабочего потока WorkerThread программы msort.cpp

На рисунке 5 представлена блок-схема алгоритма, выполняемого рабочим потоком в программе msort.cpp. Рабочие потоки используются для выполнения задач слияния на каждом этапе сортировки. Каждый поток ожидает появления задачи в общей очереди, выполняет слияние заданных диапазонов массива и сообщает о завершении задачи текущего этапа.

Рабочий поток начинает выполнение с вызова функции PopTask(&task). Если в очереди нет задач, поток блокируется на условной переменной и ожидает появления новой задачи или сигнала завершения. После получения задачи поток вызывает ProcessTask(task), внутри которой выполняется функция Merge(left, mid, right), сливающая два соседних отсортированных участка массива в один отсортированный участок. После обработки задачи вызывается FinishTask(), уменьшающая счётчик незавершённых задач текущего этапа. Затем поток возвращается к ожиданию следующей задачи.

Если задач больше нет и программа завершает работу, поток выходит из цикла и завершает выполнение.

На временной диаграмме, представленной на рисунке 6, показано взаимодействие главного потока и рабочих потоков при выполнении многопоточной сортировки слиянием. Диаграмма отражает основные моменты синхронизации: ожидание рабочими потоками появления задач на условной переменной g_cond, получение задачи из общей очереди под защитой мьютекса, выполнение слияния и ожидание главным потоком завершения текущего этапа сортировки через g_stage_cond. Обозначение R соответствует состоянию ready/running, а B – состоянию blocked.

Время	-->	-->	-->	-->	-->	-->	-->
Поток	**1. Инициализация**	**2. Создание потоков**	**3. Формирование этапа**	**4. Получение задачи**	**5. Слияние диапазонов**	**6. Завершение этапа**	**7. Завершение**
Main	R читает input.txt, создает массивы	R pthread_create	R создает задачи PushTask	B WaitStage, ждет g_stage_cond	B ожидает workers	R следующий width или завершение	R пишет output.txt, time.txt
Worker 1	–	R создан	B pthread_cond_wait (g_cond)	R/B PopTask, ждет g_mutex	R Merge(left, mid, right)	R/B FinishTask, signal g_stage_cond	Завершен
Worker 2	–	R создан	B pthread_cond_wait (g_cond)	R/B PopTask, ждет g_mutex	R Merge(left, mid, right)	R/B FinishTask, signal g_stage_cond	Завершен
Worker 3	–	R создан	B pthread_cond_wait (g_cond)	R/B PopTask, ждет g_mutex	R Merge(left, mid, right)	R/B FinishTask, signal g_stage_cond	Завершен
...	–	R создан	B pthread_cond_wait (g_cond)	R/B PopTask, ждет g_mutex	R Merge(left, mid, right)	R/B FinishTask, signal g_stage_cond	Завершен
Worker N	–	R создан	B pthread_cond_wait (g_cond)	R/B PopTask, ждет g_mutex	R Merge(left, mid, right)	R/B FinishTask, signal g_stage_cond	Завершен

Рисунок 6 – Временная диаграмма взаимодействия потоков программы msort.cpp

Таблица 6 – Зависимость времени выполнения программы msort.cpp от размера массива и количества потоков

№ теста	N	1 поток, мс	2	3	4	5	8	10	16
1	50000	58	40	32	29	31	34	18	19
2	100000	94	61	52	45	42	39	35	33
3	200000	168	103	86	72	68	57	51	48
4	5000000	4857	2528	1824	1397	1216	934	872	761
5	7500000	7149	3742	2696	2094	1783	1362	1241	1087
6	10000000	9463	4981	3618	2817	2389	1816	1694	1472
7	15000000	13928	7356	5327	4179	3541	2698	2516	2184
8	20000000	18476	9743	7062	5528	4687	3574	3319	2896

9	25000000	22941	12138	8794	6876	5823	4448	4135	3612
10	30000000	27845	14592	10581	8267	7014	5203	4968	3500

В таблице 6 представлены результаты тестирования многопоточной сортировки слиянием. В качестве входных данных использовались массивы различного размера N, а количество рабочих потоков изменялось от 1 до 16. Время выполнения указано в миллисекундах. Тестирование проводилось на процессоре AMD Ryzen 7 7735H, имеющем 8 физических ядер и 16 логических потоков.

По данным таблицы можно оценить, как увеличение количества потоков влияет на скорость выполнения алгоритма при разных размерах массива. Для малых массивов выигрыш от многопоточности выражен слабее, так как часть времени уходит на синхронизацию потоков и распределение задач. Для больших массивов параллельная обработка становится более эффективной, поэтому время выполнения заметно уменьшается при увеличении числа потоков.

Классический MergeSort формально хорошо параллелится, но в данной лабораторной он получается не самым удачным вариантом. Если разбивать массив до минимальных частей, например до отдельных элементов, то появляется слишком много мелких задач. Каждая такая задача сама по себе выполняется очень быстро, но на её создание, добавление в очередь, пробуждение потока и синхронизацию тратится время. В итоге программа начинает больше заниматься обслуживанием потоков, чем самой сортировкой.

Поэтому в реализации используется более практичный подход: сначала массив делится на достаточно крупные блоки, например по 1000 элементов. Каждый блок сортируется отдельно, а потом уже отсортированные блоки постепенно сливаются. Так количество задач становится меньше, потоки получают более “крупную” работу, и накладные расходы на синхронизацию не начинают доминировать над полезными вычислениями. Поэтому такой вариант обычно работает стабильнее и быстрее на больших массивах.

2.4 Моделирование задачи обедающих философов с использованием семафоров

Программа phil.cpp моделирует классическую задачу об обедающих философах. В программе создаётся 7 потоков: главный поток main, пять потоков философов и один управляющий поток. Каждый философ поочерёдно находится в одном из двух состояний: T – размышляет, E – ест. Изначально все философы находятся в состоянии размышления.

В данной реализации философы не берут вилки самостоятельно. Когда философ хочет есть, он отправляет запрос управляющему потоку и ожидает разрешения. Управляющий поток хранит очередь запросов и выдаёт разрешение только в том случае, если соседние философы в данный момент не едят. Это позволяет избежать ситуации взаимной блокировки, когда каждый философ взял бы одну вилку и ждал вторую.

Для синхронизации используются семафоры и мьютекс. Семафор g_manager_sem используется для пробуждения управляющего потока, когда появляется новый запрос или меняется состояние философа. Для каждого философа создаётся отдельный семафор g_permission_sem, через который управляющий поток разрешает конкретному философу перейти к состоянию еды. Мьютекс g_mutex защищает общие данные: очередь запросов, состояния философов, флаги ожидания и завершения.

Работа программы ограничена временем TOTAL, которое передаётся первым аргументом командной строки. Второй аргумент PHIL задаёт минимальное время, которое философ проводит в одном состоянии. Философ сначала размышляет не менее PHIL миллисекунд, затем отправляет запрос на еду. Если управляющий поток разрешает ему есть, философ переходит из состояния T в состояние E, ест не менее PHIL миллисекунд, после чего возвращается в состояние T.

После истечения времени TOTAL новые переходы в состояние E запрещаются. При этом философы, которые уже начали есть до завершения основного времени работы программы, должны закончить еду и вернуться в

состояние T. Благодаря этому к моменту завершения программы все философы находятся в состоянии размышления, а общее время работы не превышает TOTAL + PHIL. В таблице 7 представлены функции программы phil.cpp.

Таблица 7 – Назначение функций программы phil.cpp

Функция	Назначение
GetTimeMs()	Возвращает количество миллисекунд, прошедших с момента запуска программы. Используется для вывода времени изменения состояния философов.
SleepMs()	Приостанавливает выполнение потока на заданное количество миллисекунд. Используется для моделирования размышления и еды философов.
PrintState()	Выводит в стандартный поток строку изменения состояния философа в формате TIME:NUM:OLD->NEW.
LeftNeighbor(),RightNeighbor()	Возвращает номер левого и правого соседа философа соответственно.
CanEat()	Проверяет, может ли философ начать есть, то есть не едят ли его левый и правый соседи.
AddRequest()	Добавляет философа в очередь запросов на получение разрешения есть, если он ещё не ожидает разрешения.
PopRequest()	Извлекает первый запрос из очереди запросов управляющего потока.
TryGivePermissions()	Просматривает очередь запросов и выдаёт разрешения тем философам, которым можно есть без конфликта с соседями.
PhilosopherThread()	Поточная функция философа. Моделирует цикл: размышление, запрос разрешения, ожидание, еда, возврат к размышлению.
ManagerThread()	Поточная функция управляющего потока. Ожидает запросы философов и выдаёт разрешения на еду.
main()	Считывает аргументы командной строки, инициализирует семафоры и мьютекс, создаёт потоки, контролирует время работы программы и освобождает ресурсы.

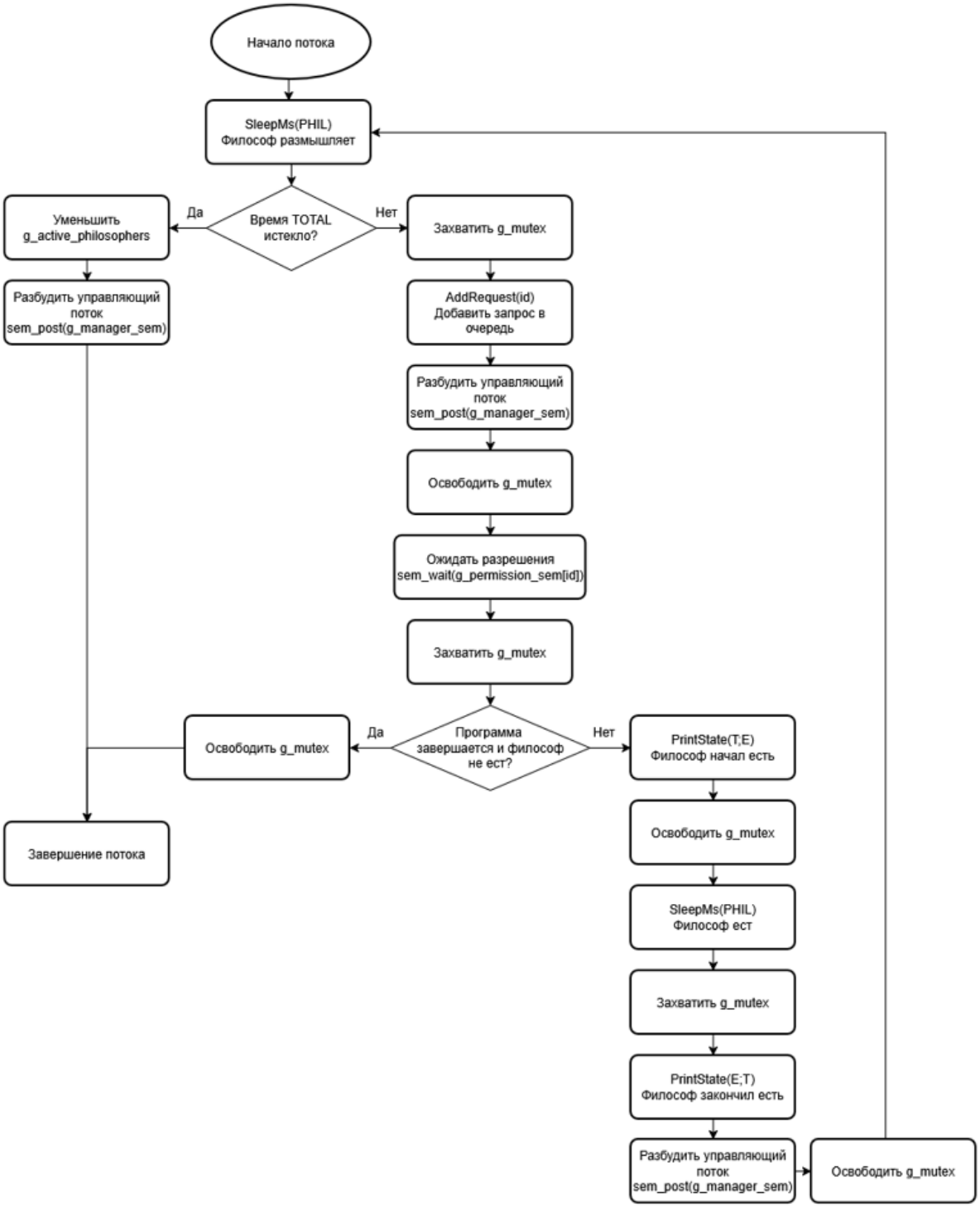


Рисунок 7 – Блок-схема алгоритма потока философа

На рисунке 7 представлена блок-схема работы одного потока философа в программе phil.cpp. После запуска поток философа сначала находится в состоянии размышления, что моделируется задержкой SleepMs(PHIL). После этого проверяется, не истекло ли общее время работы программы TOTAL. Если время истекло, философ больше не пытается перейти к еде, уменьшает

счётчик активных философов, пробуждает управляющий поток и завершает работу.

Если время TOTAL ещё не истекло, философ захватывает мьютекс g_mutex, добавляет свой запрос на еду в общую очередь с помощью функции AddRequest(id) и пробуждает управляющий поток через sem_post(g_manager_sem). Затем философ освобождает мьютекс и блокируется на своём личном семафоре g_permission_sem[id], ожидая разрешения на еду.

После получения разрешения философ снова захватывает мьютекс и проверяет, не началось ли завершение программы. Если программа завершается и философу не было выдано разрешение на еду, поток выходит из цикла и завершается. В противном случае философ меняет состояние с T на E, выводит сообщение о переходе, освобождает мьютекс и находится в состоянии еды не менее PHIL миллисекунд.

После завершения еды философ снова захватывает мьютекс, выводит переход из состояния E в T, освобождает занятые вилки через изменение общих флагов состояния и пробуждает управляющий поток. Далее цикл повторяется до тех пор, пока не истечёт время TOTAL. Такая схема позволяет философам ожидать разрешение без активного ожидания и предотвращает одновременную еду соседних философов.

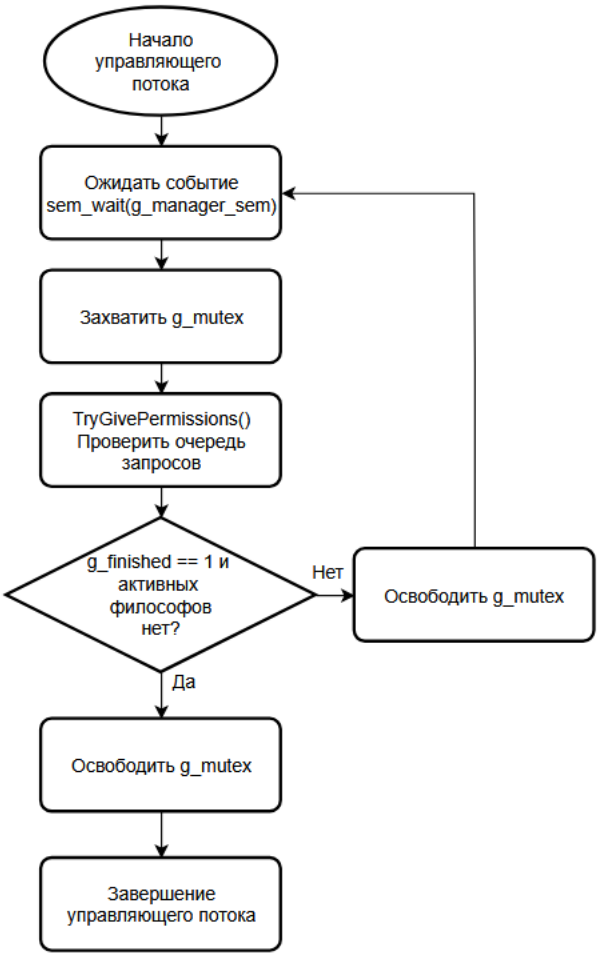


Рисунок 8 – Блок-схема алгоритма управляющего потока

На рисунке 8 показан алгоритм управляющего потока. Управляющий поток большую часть времени находится в заблокированном состоянии на семафоре g_manager_sem. Он просыпается, когда философ отправляет запрос на еду или когда один из философов заканчивает есть. После пробуждения управляющий поток проверяет очередь запросов и выдаёт разрешения тем философам, у которых оба соседа в данный момент не едят. После завершения всех потоков философов управляющий поток также завершает работу.

Время	--->	--->	--->	--->	--->	--->	--->
Поток	**1. Инициализация**	**2. Создание потоков**	**3. Ожидание запросов**	**4. Получение**	**5. Приём пищи**	**6. Освобождение вилок**	**7. Завершение**
Main	R читает аргументы TOTAL, PHIL;	R pthread_create:	B SleepMs(TOTAL)	B не выдает разрешения;	B pthread_join	R/B будит manager	R освобождает sem/mutex
Manager	—	R создан	B sem_wait(g_manager_sem)	R/B ждет g_mutex;	B ждет новых запросов	R sem_post(permission[id])	Завершен
Philosopher 1	—	R создан	R/B SleepMs(PHIL);	B/R sem_wait(permission[1]);	R PrintState(T->E);	R/B ждет g_mutex;	Завершен
Philosopher 2	—	R создан	R/B SleepMs(PHIL);	B/R sem_wait(permission[2]);	R PrintState(T->E);	R/B ждет g_mutex;	Завершен
Philosopher 3	—	R создан	R/B SleepMs(PHIL);	B/R sem_wait(permission[3]);	R PrintState(T->E);	R/B ждет g_mutex;	Завершен
...	—	R созданы	R/B SleepMs(PHIL);	B/R ждут личные семафоры	R едят после разрешения	R/B освобождают вилки;	Завершены
Philosopher 5	—	R создан	R/B SleepMs(PHIL);	B/R sem_wait(permission[5]);	R PrintState(T->E);	R/B ждет g_mutex;	Завершен

Рисунок 9 – Временная диаграмма взаимодействия потоков программы phil.cpp

На рисунке 9 представлена временная диаграмма взаимодействия потоков в программе phil.cpp. Диаграмма показывает основные этапы работы программы и состояния потоков в моменты синхронизации. Обозначение R означает, что поток выполняется или готов к выполнению, а B означает, что поток находится в заблокированном состоянии и ожидает объект синхронизации.

Главный поток Main выполняет инициализацию программы: считывает аргументы командной строки TOTAL и PHIL, создаёт мьютекс и семафоры, после чего создаёт управляющий поток и пять потоков философов. Затем главный поток ожидает истечения времени работы программы и позже ожидает завершения всех созданных потоков.

Управляющий поток Manager после создания блокируется на семафоре g_manager_sem и просыпается только тогда, когда философ отправляет запрос на еду или освобождает вилки. После пробуждения управляющий поток захватывает мьютекс g_mutex, проверяет очередь запросов и выдаёт разрешение на еду тем философам, у которых соседи в данный момент не едят. Разрешение выдаётся через личный семафор философа g_permission_sem[id].

Потоки философов после создания периодически находятся в состоянии размышления, затем отправляют запрос управляющему потоку и блокируются на своём личном семафоре разрешения. После получения разрешения философ переходит в состояние еды, выводит сообщение T->E, затем после завершения еды освобождает вилки, выводит сообщение E->T и пробуждает управляющий поток. Такая схема позволяет избежать активного ожидания, взаимной блокировки и ситуации, когда соседние философы едят одновременно.

Таблица 8 – Результаты тестирования программы phil.cpp при разных значениях TOTAL и PHIL

№ теста	TOTAL, мс	PHIL, мс	P1, E	P1, T	P2, E	P2, T	P3, E	P3, T	P4, E	P4, T	P5, E	P5, T
1	200	20	3	3	4	4	3	3	4	4	4	4
2	200	50	2	2	1	1	2	2	0	0	1	1
3	200	100	0	0	0	0	1	1	0	0	1	1
4	400	20	8	8	6	6	6	6	8	8	8	8
5	400	50	3	3	3	3	3	3	3	3	2	2
6	400	100	1	1	1	1	1	1	2	2	1	1
7	1000	20	19	19	20	20	19	19	17	17	19	19
8	1000	50	8	8	7	7	8	8	7	7	8	8
9	1000	100	3	3	4	4	3	3	4	4	4	4
10	2000	20	37	37	36	36	34	34	37	37	38	38
11	2000	50	16	16	15	15	15	15	14	14	14	14
12	2000	100	8	8	8	8	7	7	7	7	8	8
13	4000	20	69	69	72	72	69	69	70	70	69	69
14	4000	50	30	30	30	30	30	30	29	29	29	29
15	4000	100	16	16	16	16	16	16	14	14	16	16
16	20000	20	364	364	370	370	366	366	369	369	363	363
17	20000	50	149	149	153	153	148	148	151	151	149	149
18	20000	100	70	70	73	73	79	79	77	77	75	75

В таблице 8 представлены результаты тестирования программы phil.cpp, моделирующей задачу обедающих философов. Для каждого испытания указаны значения TOTAL и PHIL, а также количество квантов еды E и размышления T для каждого из пяти философов. Под квантом еды понимается один переход философа в состояние E, а под квантом размышления - один переход в состояние T.

По результатам тестирования видно, что распределение квантов еды между философами в целом является равномерным. При больших значениях времени работы программы, например при TOTAL = 20000 мс, количество переходов в состояние еды у разных философов отличается незначительно. Это показывает, что управляющий поток выдаёт разрешения достаточно справедливо и не допускает бесконечного голодания отдельных философов.

При малых значениях TOTAL, например 200 мс и 400 мс, различия между философами выражены сильнее. Это связано с тем, что программа работает слишком короткое время, и не все философы успевают одинаковое количество раз получить разрешение на еду. Также на результат влияет

значение PHIL: чем больше минимальное время еды и размышления, тем меньше общее количество переходов между состояниями успевает произойти за время теста.

Во всех проведённых испытаниях количество квантов E и T у каждого философа совпадает. Это означает, что каждый философ, начавший есть, затем корректно завершал приём пищи и возвращался в состояние размышления. Следовательно, к моменту завершения программы философы не оставались в состоянии еды, а алгоритм корректно завершал работу потоков.

Таким образом, реализованный алгоритм с управляющим потоком обеспечивает отсутствие взаимных блокировок и достаточно равномерное распределение доступа к вилкам между философами.

2.5 Поиск расстановок шахматных королей на доске

Программа chess.cpp решает задачу подсчёта количества возможных расстановок королей на шахматной доске размером N x N. На доске уже может быть размещено K королей, координаты которых считываются из файла input.txt. Необходимо расставить ещё L королей так, чтобы ни один король не находился под боем другого. Так как по варианту задания рассматривается фигура “король”, запрещёнными являются все соседние клетки вокруг уже поставленного короля: по горизонтали, вертикали и диагонали.

Для исключения повторного подсчёта одинаковых расстановок клетки доски рассматриваются в фиксированном порядке: слева направо и сверху вниз. Каждая клетка получает линейный номер от 0 до N*N - 1. При рекурсивном переборе следующий король может ставиться только в клетки с номером больше, чем у предыдущей рассмотренной позиции. Благодаря этому одна и та же расстановка не считается несколько раз в разном порядке.

В начале программа считывает входные данные, создаёт начальную доску и отмечает на ней уже установленные короли. Затем формируется набор начальных задач для рабочего пула потоков. Каждая задача содержит копию текущей доски, позицию, с которой нужно продолжать перебор, и количество королей, которые ещё необходимо поставить. Рабочие потоки получают

задачи из общей очереди и выполняют рекурсивный поиск решений для своей части пространства вариантов.

В процессе рекурсии программа перебирает клетки доски, проверяет, можно ли поставить короля в текущую клетку, и если можно, создаёт копию доски с новым королём. После этого рекурсивно продолжается поиск для оставшегося количества фигур. Если все требуемые короли успешно расставлены, найденная расстановка считается одним решением, и счётчик решений увеличивается.

Для синхронизации потоков используются объекты Windows API. Событие g_start_event позволяет запустить все рабочие потоки одновременно после создания пула и начала измерения времени. Мьютекс g_task_mutex защищает выдачу задач потокам, чтобы одну и ту же задачу не взяли несколько потоков. Мьютекс g_answer_mutex защищает общий счётчик найденных решений. После завершения всех потоков программа записывает количество решений в файл output.txt, а время работы алгоритма – в файл time.txt. Функции chess.cpp представлены в таблице 9.

Таблица – 9 Назначение функций программы chess.cpp

Функция	Назначение
GetIndex()	Переводит координаты клетки (x, y) в линейный индекс массива доски.
CopyBoard()	Создаёт копию текущей доски. Используется при рекурсивном переборе, чтобы каждая ветка поиска работала со своим состоянием доски.
CanPlace()	Проверяет, можно ли поставить короля в указанную клетку. Возвращает запрет, если клетка занята или рядом уже стоит другой король.
Search()	Выполняет рекурсивный перебор вариантов расстановки королей. Если все короли расставлены, возвращает найденное решение.
GetNextTask()	Выдаёт рабочему потоку следующую задачу из общего массива задач. Доступ защищается мьютексом.

AddAnswer()	Добавляет локальный результат потока к общему счётчику решений g_answer. Доступ защищается мьютексом.
WorkerThread()	Поточная функция рабочего потока. Ожидает стартовое событие, получает задачи, выполняет рекурсивный поиск и добавляет найденные решения в общий ответ.
main()	Считывает входные данные, создаёт начальную доску, формирует задачи, создаёт потоки, запускает вычисления, ожидает завершения потоков и записывает результаты в файлы output.txt и time.txt.

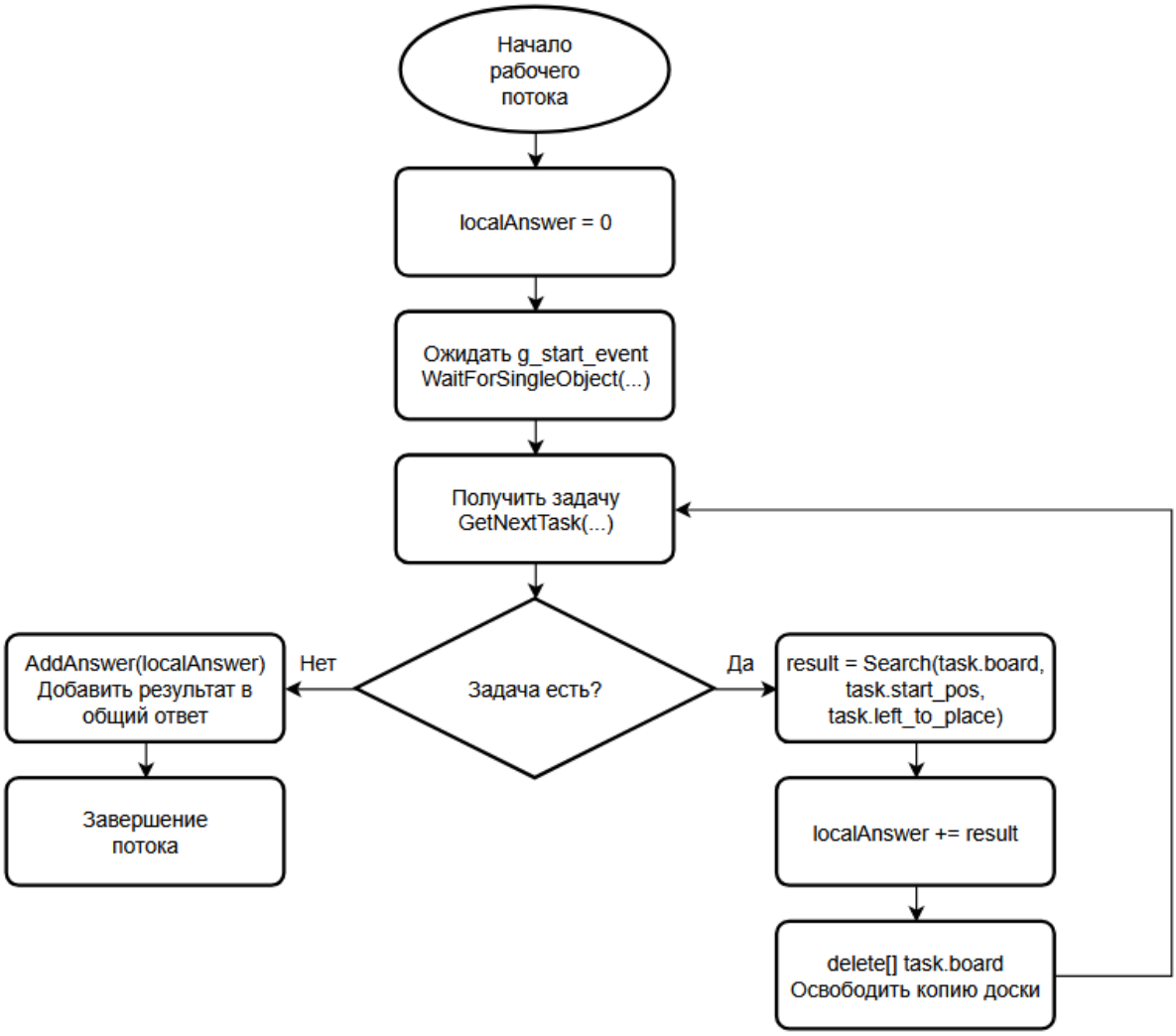


Рисунок 10 – Блок-схема алгоритма рабочего потока программы chess.cpp

На рисунке показан алгоритм работы одного рабочего потока в программе chess.cpp. После запуска поток сначала создаёт локальный счётчик найденных решений localAnswer, затем ожидает стартовое событие g_start_event. Это нужно для того, чтобы все рабочие потоки начали вычисления одновременно после начала измерения времени.

После получения стартового сигнала поток входит в цикл обработки задач. На каждой итерации он вызывает функцию GetNextTask(), которая выдаёт следующую задачу из общего массива задач. Если задача есть, поток запускает рекурсивный поиск Search() для своей части вариантов расстановки королей и прибавляет найденное количество решений к локальному счётчику. После завершения обработки задачи копия доски, связанная с этой задачей, освобождается.

Если свободных задач больше нет, поток выходит из цикла и с помощью функции AddAnswer() добавляет свой локальный результат к общему счётчику g_answer. После этого рабочий поток завершает выполнение. Использование локального счётчика позволяет уменьшить количество обращений к общему счётчику и, соответственно, сократить число блокировок мьютекса.

Время	--->	--->	--->	--->	--->	--->	--->
Поток	**1.**	**2. Создание**	**3. Ожидание старта**	**4. Получение**	**5. Поиск решений**	**6. Запись ответа**	**7.**
Main	R читает input.txt;	R CreateThread	R старт таймера;	B WaitForSingleObject	B ожидает завершения	B ожидает worker	R пишет output.txt,
Worker 1	—	R создан	B WaitForSingleObject	R/B ждет g_task_mutex;	R Search(board, start, left)	R/B ждет g_answer_mutex;	Завершен
Worker 2	—	R создан	B WaitForSingleObject	R/B ждет g_task_mutex;	R Search(board, start, left)	R/B ждет g_answer_mutex;	Завершен
Worker 3	—	R создан	B WaitForSingleObject	R/B ждет g_task_mutex;	R Search(board, start, left)	R/B ждет g_answer_mutex;	Завершен
...	—	R созданы	B WaitForSingleObject	R/B ждут g_task_mutex;	R Search(...)	R/B ждут g_answer_mutex;	Завершены
Worker N	—	R создан	B WaitForSingleObject	R/B ждет g_task_mutex;	R Search(board, start, left)	R/B ждет g_answer_mutex;	Завершен

Рисунок 11 – Временная диаграмма взаимодействия потоков программы chess.cpp

На диаграмме показаны состояния главного потока и рабочих потоков в основные моменты синхронизации. Главный поток выполняет инициализацию, создаёт объекты синхронизации и рабочие потоки, после чего запускает вычисления через SetEvent(g_start_event). Рабочие потоки до этого момента находятся в заблокированном состоянии на WaitForSingleObject(g_start_event). Далее они получают задачи с защитой

через g_task_mutex, выполняют рекурсивный поиск решений и в конце добавляют локальный результат в общий счётчик g_answer с защитой через g_answer_mutex.

Таблица 10 – Результаты тестирования программы chess.cpp при разных размерах задачи и количестве потоков

№ теста	N, L, K	1 поток, мс	2	3	4	5	8	10	16
1	4 1 0	15	0	0	0	0	0	0	0
2	8 4 0	47	15	16	0	0	0	0	0
3	10 3 0	16	0	0	0	0	0	0	0
4	10 4 0	266	125	94	79	62	32	32	31
5	10 5 0	3859	1937	1296	984	797	578	531	453
6	10 6 0	42750	21391	14281	10812	8797	5938	5515	5031
7	11 3 0	31	0	0	0	0	0	0	0
8	11 4 0	609	297	203	141	125	109	78	78
9	11 5 0	11250	5797	3765	2813	2265	1516	1500	1312
10	11 6 0	175750	125781	86391	66250	33094	36828	20485	18078

Тестирование программы chess.cpp проводилось на процессоре AMD Ryzen 7 7735H. В таблице 10 показана зависимость времени выполнения алгоритма от размера задачи и количества рабочих потоков. В качестве параметров задачи использовались размер доски N, количество расставляемых королей L и количество уже установленных фигур K.

По результатам видно, что при увеличении сложности задачи время выполнения программы значительно возрастает. Особенно заметно это при переходе к наборам 10 6 0 и 11 6 0, где количество возможных вариантов расстановки становится намного больше. Это объясняется тем, что программа выполняет полный перебор допустимых расстановок, и при увеличении числа фигур резко растёт количество ветвей рекурсивного поиска.

Использование нескольких потоков позволяет заметно сократить время выполнения для трудоёмких тестов. Например, для набора 10 6 0 время уменьшается с 42750 мс при одном потоке до 5031 мс при 16 потоках. Для набора 11 6 0 также наблюдается существенное ускорение при увеличении числа потоков.

При этом ускорение не является строго пропорциональным количеству потоков. В некоторых случаях при увеличении числа потоков результат может улучшаться неравномерно или даже немного ухудшаться. Это связано с накладными расходами на синхронизацию, выдачу задач потокам, конкуренцией за общие мьютексы и особенностями планирования потоков операционной системой.

Для маленьких тестов многопоточность почти не даёт преимущества, так как сама задача выполняется слишком быстро. В таких случаях время на создание и синхронизацию потоков становится сопоставимо с временем вычислений, поэтому результаты могут быть равны нулю миллисекунд или отличаться незначительно.

Таким образом, многопоточный алгоритм наиболее эффективен на больших задачах, где объём перебора достаточно велик. Для малых входных данных использование большого количества потоков нецелесообразно, а для сложных задач параллельное выполнение позволяет существенно сократить время поиска решений.

ВЫВОД

В ходе лабораторной работы были изучены основные способы создания и синхронизации потоков в операционных системах Windows и Linux. Были реализованы многопоточные программы для параллельных вычислений, быстрой сортировки, сортировки слиянием, задачи обедающих философов и задачи поиска расстановок шахматных королей.

В программах использовались разные механизмы синхронизации: мьютексы, семафоры, события, критические секции и условные переменные. На практике было показано, что выбор объекта синхронизации зависит от конкретной задачи. Мьютексы и критические секции удобно использовать для защиты общих данных, семафоры – для организации очереди задач и ограничения доступа, события – для одновременного запуска потоков, а условные переменные – для ожидания изменения состояния без активного ожидания.

Также в работе был реализован пул потоков. Потоки создавались один раз в начале работы программы, после чего получали задачи из общей очереди. Такой подход позволяет избежать постоянного создания и уничтожения потоков и делает выполнение программы более эффективным.

По результатам тестирования было установлено, что многопоточность даёт заметное ускорение на достаточно больших задачах. При малом объёме входных данных ускорение может быть слабым или отсутствовать, так как накладные расходы на синхронизацию и переключение потоков становятся сопоставимы с временем самих вычислений. При увеличении размера задачи параллельное выполнение становится эффективнее, однако ускорение не всегда пропорционально количеству потоков из-за конкуренции за общие ресурсы и особенностей планирования потоков операционной системой.

В задаче обедающих философов была реализована схема с управляющим потоком, которая позволила избежать взаимной блокировки и бесконечного голодания философов. Тестирование показало, что доступ к

ресурсам распределяется достаточно равномерно, особенно при большой длительности эксперимента.

Таким образом, цель лабораторной работы была достигнута: были получены практические навыки разработки многопоточных программ и использования различных средств синхронизации потоков в Windows и Linux.

Приложение А «expr.cpp»

```
// Программа считает разложение числа N

#include <windows.h> // Нужно для Windows API
#include <stdio.h>

#define MAX_THREADS 1024

int g_threads_count = 0; // Кол-во потоков
int g_n = 0;             // Число N из input.txt

HANDLE g_start_event = NULL;  // Событие, через которое главный поток одновременно запускает рабочих
HANDLE g_task_mutex = NULL;   // Мьютекс для выдачи задач, чтобы два потока не взяли один и тот же first
HANDLE g_result_mutex = NULL; // Мьютекс для добавления результата в g_answer

int g_next_first = 0;             // Следующее первое слагаемое, которое надо выдать потоку
unsigned long long g_answer = 0;  // Общий ответ

unsigned long long CountParts(int sum, int maxPart)
{
    if (sum == 0) return 1;   // Если сумма уже набрана, то значит найден один готовый вариант

    unsigned long long count = 0; // кол-во вариантов

    int limit = maxPart; // наше число maxPart не должно быть больше sum
    if (limit>sum) limit = sum;

    for (int part = limit; part >=1 ; part--){ // Какое число поставим следующим в сумму
        count += CountParts(sum - part, part);
    }

    return count;
}

unsigned long long CountForFirst (int first) // Одна работа потока.
{
    return CountParts(g_n - first, first);
}

int GetNextFirst()
{
    int first = 0;
```

```
    WaitForSingleObject(g_task_mutex, INFINITE); // Захватываем
замок, любой кто зайдёт, ждёт сколь угодно

    if (g_next_first>0)
    {
        first = g_next_first;
        g_next_first--;
    }

    ReleaseMutex(g_task_mutex);

    return first;
}

void AddAnswer(unsigned long long value)
{
    WaitForSingleObject(g_result_mutex, INFINITE);

    g_answer += value;

    ReleaseMutex(g_result_mutex);
}

DWORD WINAPI WorkerThread(void* param)
{
    unsigned long long localAnswer = 0;

    WaitForSingleObject(g_start_event, INFINITE);

    while (1)
    {
        int first = GetNextFirst();

        if (first <= 0)
            break;

        localAnswer += CountForFirst(first);
    }

    AddAnswer(localAnswer);

    return 0;
}

int main()
{
    FILE* input = fopen("input.txt", "r");
    /*  Входные данные: файл input.txt.
        Первая строка — натуральное число T, количество потоков
рабочего пула для решения задачи.
        Вторая строка — натуральное число N (2 <= N <= 100)
    */
```

```
    if (input == NULL)
    {
        return 1;
    }

    fscanf(input, "%d", &g_threads_count); // Считываем кол-во
потоков
    fscanf(input, "%d", &g_n);             // Считываем само
натуральное число

    fclose(input);

    g_next_first = g_n - 1;                // Начинаем с цифры
меньше N

    g_start_event = CreateEvent(NULL, TRUE, FALSE, NULL); //
Общая кнопка start для всех рабочих потоков
    /* CreateEvent(права_доступа, тип_сброса,
начальное_состояние, имя);
       NULL значит: используем стандартные настройки.
       TRUE значит manual-reset event, то есть событие с ручным
сбросом.
       FALSE значит: событие сначала не сигнальное.
       Четвёртый параметр — имя события
       Создать безымянное событие, сначала выключенное, с ручным
сбросом,
       чтобы потом одним SetEvent разбудить все потоки. */

    g_task_mutex = CreateMutex(NULL, FALSE, NULL); // Создать
свободный безымянный мьютекс.
    // Чтобы два потока не взяли одну и ту же задачу.
    /* CreateMutex(права_доступа, захватить_сразу, имя);
       NULL - Стандартные настройки безопасности.
       FALSE - Создавший поток не захватывает мьютекс сразу. */
    g_result_mutex = CreateMutex(NULL, FALSE, NULL);
    // Чтобы два потока одновременно не прибавляли к общему
ответу.

    if (g_start_event == NULL || g_task_mutex == NULL ||
g_result_mutex == NULL)
    {
        return 1;
    }

    HANDLE* threads = new HANDLE[g_threads_count]; // массив
описателей потоков

    for (int i = 0; i < g_threads_count; i++)
    {
        threads[i] = CreateThread(NULL, 0, WorkerThread, NULL,
0, NULL);

        if (threads[i] == NULL)
```

```
        {
            return 1;
        }
    }

    DWORD startTime = GetTickCount(); // Запоминаем время старта
алгоритма

    SetEvent(g_start_event); // Будим рабочие потоки

    for (int i = 0; i < g_threads_count; i++)
    {
        WaitForSingleObject(threads[i], INFINITE);
    }


    DWORD finishTime = GetTickCount(); // Останавлививаем таймер
и считаем время работы алгоритма
    DWORD workTime = finishTime - startTime;

    FILE* output = fopen("output.txt", "w");
    if (output != NULL)
    {
        fprintf(output, "%d\n", g_threads_count);
        fprintf(output, "%d\n", g_n);
        fprintf(output, "%llu\n", g_answer);
        fclose(output);
    }

    FILE* timeFile = fopen("time.txt", "w");
    if (timeFile != NULL)
    {
        fprintf(timeFile, "%lu\n", (unsigned long)workTime);
        fclose(timeFile);
    }

    for (int i = 0; i < g_threads_count; i++)
    {
        CloseHandle(threads[i]);
    }

    CloseHandle(g_start_event);
    CloseHandle(g_task_mutex);
    CloseHandle(g_result_mutex);

    delete[] threads;

    return 0;
}
```

Приложение Б «qsort.cpp»

```
#include <windows.h>
#include <stdio.h>

#define SMALL_SIZE 1000   // Если массив <=1000, поток
отсортирует его сам
#define MAX_THREADS 1024

struct SortTask // Структура задачи. Отсортировать массив от
left до right
{
    int left;
    int right;
};

int g_threads_count = 0; // Кол-во рабочих потоков
int g_n = 0;             // Размер массива
int* g_array = NULL;     // Массив чисел

SortTask* g_tasks = NULL; // массив задач
int g_task_head = 0;      // откуда забираем задачу
int g_task_tail = 0;      // куда кладём новую задачу
int g_task_count = 0;     // сколько задач сейчас в очереди и
никем не взяты

int g_work_count = 0;     // кол-во задач, которые ещё не
завершены
int g_finish = 0;         // Флаг завершения

CRITICAL_SECTION g_cs;
HANDLE g_task_sem = NULL;

void PushTask(int left, int right) // Функция добавления задачи
в очередь
{
    if (left >= right) // Если в диапазоне 0 или 1 элемент,
сортировать нечего.
        return;

    EnterCriticalSection(&g_cs); // Заходим в крит. секцию, так
как меняем общую очередь

    // Кладём новую задачу в конец очереди.
    g_tasks[g_task_tail].left = left;
    g_tasks[g_task_tail].right = right;

    g_task_tail++;
    g_task_count++;
    g_work_count++;

    LeaveCriticalSection(&g_cs);
```

```
    // Сообщаем семафору: появилась одна задача, можно разбудить
поток
    ReleaseSemaphore(g_task_sem, 1, NULL);
}

int PopTask(SortTask* task)
{
    WaitForSingleObject(g_task_sem, INFINITE); // Поток спит,
пока нет задач.

    EnterCriticalSection(&g_cs); // Читаем/меняем очередь

    if (g_finish) // Если программа уже закончила сортировку,
поток выходит
    {
        LeaveCriticalSection(&g_cs);
        return 0;
    }

    if (g_task_count == 0)
    {
        LeaveCriticalSection(&g_cs);
        return 0;
    }

    // Берём задачу из очереди.
    task->left = g_tasks[g_task_head].left;
    task->right = g_tasks[g_task_head].right;

    g_task_head++;
    g_task_count--;

    LeaveCriticalSection(&g_cs);

    return 1;
}

void FinishTask()
{
    EnterCriticalSection(&g_cs);

    g_work_count--;

    if (g_work_count == 0)
    {
        g_finish = 1;
        ReleaseSemaphore(g_task_sem, g_threads_count, NULL); //
Будим все рабочие потоки, которые могут спать на семафоре:
    }

    LeaveCriticalSection(&g_cs);
}
```

```
void Swap(int* a, int* b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

void SimpleQuickSort(int left, int right)
{
    int i = left;
    int j = right;
    int pivot = g_array[(left + right) / 2];

    while (i <= j)
    {
        while (g_array[i] < pivot)
            i++;

        while (g_array[j] > pivot)
            j--;

        if (i <= j)
        {
            Swap(&g_array[i], &g_array[j]);
            i++;
            j--;
        }
    }

    if (left < j)
        SimpleQuickSort(left, j);

    if (i < right)
        SimpleQuickSort(i, right);
}

void ProcessTask(SortTask task)
{
    int left = task.left;
    int right = task.right;

    if (right - left + 1 <= SMALL_SIZE)
    {
        SimpleQuickSort(left, right);
        return;
    }

    int i = left;
    int j = right;
    int pivot = g_array[(left + right) / 2];

    while (i <= j)
    {
```

```
        while (g_array[i] < pivot)
            i++;

        while (g_array[j] > pivot)
            j--;

        if (i <= j)
        {
            Swap(&g_array[i], &g_array[j]);
            i++;
            j--;
        }
    }

    PushTask(left, j);
    PushTask(i, right);
}

DWORD WINAPI WorkerThread(void* param)
{
    while (1)
    {
        SortTask task;

        if (!PopTask(&task))
            break;

        ProcessTask(task);

        FinishTask();
    }

    return 0;
}

int main()
{
    FILE* input = fopen("input.txt", "r");
    if (input == NULL)
    {
        return 1;
    }

    fscanf(input, "%d", &g_threads_count); // Кол-во потоков
    fscanf(input, "%d", &g_n);             // Размер массива

    if (g_threads_count <= 0 || g_threads_count > MAX_THREADS ||
g_n <= 0)
    {
        fclose(input);
        return 1;
    }
```

```
    g_array = new int[g_n]; // Выделяем память под массив

    for (int i = 0; i < g_n; i++) // Читаем все числа
    {
        fscanf(input, "%d", &g_array[i]);
    }

    fclose(input);

    g_tasks = new SortTask[g_n]; // задач сортировки не должно
быть больше, чем элементов массива

    // Создаём крит. секцию. Она будет защищать очередь задач,
g_work_count, g_finish
    InitializeCriticalSection(&g_cs);

    int sem_max = g_n; // Значение семафора равно кол-ву
элементов задач, к-ое может быть
    if (sem_max < g_threads_count) // Если оно меньше кол-ва
потоков, приравниваем к кол-ву потоков
        sem_max = g_threads_count;

    g_task_sem = CreateSemaphore(NULL, 0, sem_max, NULL); //
Создаём семафор
    /* NULL - стандартные настройки безопасности
        0 - начальное значение семафора ( сначала задач нет,
потоки будут спать)
        sem_max - максимальное значение семафора
        NULL - семафор без имени
        Когда появляется задача, мы увеличиваем семафор через
ReleaseSemaphore, и один поток просыпается.

    */
    if (g_task_sem == NULL)
    {
        return 1;
    }

    HANDLE* threads = new HANDLE[g_threads_count];

    for (int i = 0; i < g_threads_count; i++)
    {
        threads[i] = CreateThread(NULL, 0, WorkerThread, NULL,
0, NULL);

        if (threads[i] == NULL)
        {
            return 1;
        }
    }

    DWORD startTime = GetTickCount();
```

```
if (g_n > 1)
{
    PushTask(0, g_n - 1);
}
else
{
    EnterCriticalSection(&g_cs);
    g_finish = 1;
    LeaveCriticalSection(&g_cs);
    ReleaseSemaphore(g_task_sem, g_threads_count, NULL);
}

for (int i = 0; i < g_threads_count; i++)
{
    WaitForSingleObject(threads[i], INFINITE);
}

DWORD finishTime = GetTickCount();
DWORD workTime = finishTime - startTime;

FILE* output = fopen("output.txt", "w");
if (output != NULL)
{
    fprintf(output, "%d\n", g_threads_count);
    fprintf(output, "%d\n", g_n);

    for (int i = 0; i < g_n; i++)
    {
        if (i > 0)
            fprintf(output, " ");

        fprintf(output, "%d", g_array[i]);
    }

    fprintf(output, "\n");

    fclose(output);
}

FILE* timeFile = fopen("time.txt", "w");
if (timeFile != NULL)
{
    fprintf(timeFile, "%lu\n", (unsigned long)workTime);
    fclose(timeFile);
}

for (int i = 0; i < g_threads_count; i++)
{
    CloseHandle(threads[i]);
}

CloseHandle(g_task_sem);
DeleteCriticalSection(&g_cs);
```

```
    delete[] threads;
    delete[] g_tasks;
    delete[] g_array;

    return 0;
}
```

Приложение В «msort.cpp»

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SMALL_SIZE 1000
#define MAX_THREADS 1024

struct SortTask
{
    int left;
    int mid;
    int right;
    int type; // 0 - sort small block, 1 - merge two sorted
blocks
};

int g_threads_count = 0;
int g_n = 0;

int* g_array = NULL;
int* g_temp = NULL;

SortTask* g_tasks = NULL;
int g_task_head = 0;
int g_task_tail = 0;
int g_task_count = 0;

int g_finish = 0;
int g_stage_tasks = 0;

pthread_t* g_threads = NULL;
pthread_mutex_t g_mutex;
pthread_cond_t g_cond;
pthread_cond_t g_stage_cond;

void PushTask(int left, int mid, int right, int type)
{
    g_tasks[g_task_tail].left = left;
    g_tasks[g_task_tail].mid = mid;
    g_tasks[g_task_tail].right = right;
    g_tasks[g_task_tail].type = type;

    g_task_tail++;
    g_task_count++;
    g_stage_tasks++;

    pthread_cond_signal(&g_cond);
}

int PopTask(SortTask* task)
```

```
{
    pthread_mutex_lock(&g_mutex);

    while (g_task_count == 0 && !g_finish)
    {
        pthread_cond_wait(&g_cond, &g_mutex);
    }

    if (g_finish)
    {
        pthread_mutex_unlock(&g_mutex);
        return 0;
    }

    task->left = g_tasks[g_task_head].left;
    task->mid = g_tasks[g_task_head].mid;
    task->right = g_tasks[g_task_head].right;
    task->type = g_tasks[g_task_head].type;

    g_task_head++;
    g_task_count--;

    pthread_mutex_unlock(&g_mutex);

    return 1;
}

void FinishTask()
{
    pthread_mutex_lock(&g_mutex);

    g_stage_tasks--;

    if (g_stage_tasks == 0)
    {
        pthread_cond_signal(&g_stage_cond);
    }

    pthread_mutex_unlock(&g_mutex);
}

void SimpleSort(int left, int right)
{
    for (int i = left + 1; i <= right; i++)
    {
        int value = g_array[i];
        int j = i - 1;

        while (j >= left && g_array[j] > value)
        {
            g_array[j + 1] = g_array[j];
            j--;
        }
```

```
        g_array[j + 1] = value;
    }
}

void Merge(int left, int mid, int right)
{
    int i = left;
    int j = mid + 1;
    int k = left;

    while (i <= mid && j <= right)
    {
        if (g_array[i] <= g_array[j])
        {
            g_temp[k] = g_array[i];
            i++;
        }
        else
        {
            g_temp[k] = g_array[j];
            j++;
        }

        k++;
    }

    while (i <= mid)
    {
        g_temp[k] = g_array[i];
        i++;
        k++;
    }

    while (j <= right)
    {
        g_temp[k] = g_array[j];
        j++;
        k++;
    }

    for (i = left; i <= right; i++)
    {
        g_array[i] = g_temp[i];
    }
}

void ProcessTask(SortTask task)
{
    if (task.type == 0)
        SimpleSort(task.left, task.right);
    else
        Merge(task.left, task.mid, task.right);
```

```
}

void* WorkerThread(void* param)
{
    while (1)
    {
        SortTask task;

        if (!PopTask(&task))
            break;

        ProcessTask(task);

        FinishTask();
    }

    return 0;
}

unsigned long ToMs(struct timespec* tm)
{
    return (unsigned long)tm->tv_sec * 1000 +
           (unsigned long)tm->tv_nsec / 1000000;
}

void WaitStage()
{
    pthread_mutex_lock(&g_mutex);

    while (g_stage_tasks > 0)
    {
        pthread_cond_wait(&g_stage_cond, &g_mutex);
    }

    g_task_head = 0;
    g_task_tail = 0;
    g_task_count = 0;

    pthread_mutex_unlock(&g_mutex);
}

int main()
{
    FILE* input = fopen("input.txt", "r");
    if (input == NULL)
    {
        return 1;
    }

    fscanf(input, "%d", &g_threads_count);
    fscanf(input, "%d", &g_n);
```

```
    if (g_threads_count <= 0 || g_threads_count > MAX_THREADS ||
g_n <= 0)
    {
        fclose(input);
        return 1;
    }

    g_array = new int[g_n];
    g_temp = new int[g_n];
    g_tasks = new SortTask[g_n];
    g_threads = new pthread_t[g_threads_count];

    for (int i = 0; i < g_n; i++)
    {
        fscanf(input, "%d", &g_array[i]);
    }

    fclose(input);

    pthread_mutex_init(&g_mutex, 0);
    pthread_cond_init(&g_cond, 0);
    pthread_cond_init(&g_stage_cond, 0);

    for (int i = 0; i < g_threads_count; i++)
    {
        pthread_create(&g_threads[i], 0, WorkerThread, 0);
    }

    struct timespec startTime;
    struct timespec finishTime;

    clock_gettime(CLOCK_REALTIME, &startTime);

    pthread_mutex_lock(&g_mutex);

    for (int left = 0; left < g_n; left += SMALL_SIZE)
    {
        int right = left + SMALL_SIZE - 1;

        if (right >= g_n)
            right = g_n - 1;

        PushTask(left, 0, right, 0);
    }

    pthread_mutex_unlock(&g_mutex);

    WaitStage();

    for (int width = SMALL_SIZE; width < g_n; width *= 2)
    {
        pthread_mutex_lock(&g_mutex);
```

```
            for (int left = 0; left < g_n; left += width * 2)
            {
                int mid = left + width - 1;
                int right = left + width * 2 - 1;

                if (mid >= g_n - 1)
                    continue;

                if (right >= g_n)
                    right = g_n - 1;

                PushTask(left, mid, right, 1);
            }

            pthread_mutex_unlock(&g_mutex);

            WaitStage();
        }

        clock_gettime(CLOCK_REALTIME, &finishTime);

        pthread_mutex_lock(&g_mutex);
        g_finish = 1;
        pthread_cond_broadcast(&g_cond); // будим все потоки
        pthread_mutex_unlock(&g_mutex);

        for (int i = 0; i < g_threads_count; i++)
        {
            pthread_join(g_threads[i], 0); // main ждёт пока
завершатся все потоки
        }

        unsigned long workTime = ToMs(&finishTime) -
ToMs(&startTime);

        FILE* output = fopen("output.txt", "w");
        if (output != NULL)
        {
            fprintf(output, "%d\n", g_threads_count);
            fprintf(output, "%d\n", g_n);

            for (int i = 0; i < g_n; i++)
            {
                if (i > 0)
                    fprintf(output, " ");

                fprintf(output, "%d", g_array[i]);
            }

            fprintf(output, "\n");
            fclose(output);
        }
```

```
    FILE* timeFile = fopen("time.txt", "w");
    if (timeFile != NULL)
    {
        fprintf(timeFile, "%lu\n", workTime);
        fclose(timeFile);
    }

    pthread_cond_destroy(&g_stage_cond);
    pthread_cond_destroy(&g_cond);
    pthread_mutex_destroy(&g_mutex);

    delete[] g_threads;
    delete[] g_tasks;
    delete[] g_temp;
    delete[] g_array;

    return 0;
}
```

Приложение Г «phil.cpp»

```
#include <pthread.h>  // для потоков Linux
#include <semaphore.h>  // для семафоров
#include <stdio.h>   // для atoi
#include <stdlib.h>
#include <unistd.h> // для usleep (сон в микросекундах)
#include <time.h>   // для измерения времени

#define PHIL_COUNT 5       // Философов всегда 5
#define STATE_THINKING 'T'
#define STATE_EATING 'E'

int g_total_time = 0; // TOTAL из командной строки
int g_phil_time = 0;  // PHIL из командной строки

struct timespec g_start_time; // время старта программы

pthread_t g_philosophers[PHIL_COUNT]; // 5 потоков философов
pthread_t g_manager; // поток управляющего

sem_t g_manager_sem; // на нём спит управляющий поток
sem_t g_permission_sem[PHIL_COUNT]; // отдельный семафор для
каждого философа

pthread_mutex_t g_mutex;
/* Мьютекс для общих данных:
   очередь запросов,
   состояния философов
   кто ждёт
   кто ест
   флаги завершения
*/

char g_state[PHIL_COUNT]; // Текущее состояние каждого философа
```

```
int g_request_queue[PHIL_COUNT]; // Массив очереди
int g_queue_head = 0;  // откуда забираем запрос
int g_queue_tail = 0;  // куда кладём новый запрос
int g_queue_count = 0; // сколько запросов сейчас лежит в
очереди

int g_waiting[PHIL_COUNT]; // массив флагов, ждёт ли филосов уже
разрешения есть
// 1 - философ попросил есть и ждёт разрешения
// 0 - философ сейчас не ждёт разрешения
int g_eating[PHIL_COUNT]; // чтобы один и тот же философ не
попал в очередь несколько раз

int g_finished = 0;
int g_active_philosophers = PHIL_COUNT;

int g_eat_count[PHIL_COUNT];
int g_think_count[PHIL_COUNT];

long GetTimeMs() // Сколько мс прошло от старта программы
{
    struct timespec now;
    clock_gettime(CLOCK_REALTIME, &now);

    long sec = now.tv_sec - g_start_time.tv_sec;
    long nsec = now.tv_nsec - g_start_time.tv_nsec;

    return sec * 1000 + nsec / 1000000;
}

void SleepMs(int ms) // Философ думает минимум PHIL time
{
    usleep(ms * 1000);
}

void PrintState(int id, char oldState, char newState)
```

```
{
    if (oldState == STATE_THINKING && newState == STATE_EATING)
        g_think_count[id]++;

    if (oldState == STATE_EATING && newState == STATE_THINKING)
        g_eat_count[id]++;

    printf("%ld:%d:%c->%c\n", GetTimeMs(), id + 1, oldState,
newState);
    fflush(stdout);
}

int LeftNeighbor(int id) // Узнать левого соседа
{
    return (id + PHIL_COUNT - 1) % PHIL_COUNT;
}

int RightNeighbor(int id) // Узнать правого соседа
{
    return (id + 1) % PHIL_COUNT;
}

int CanEat(int id) // Может ли текущий философ есть
{
    int left = LeftNeighbor(id);
    int right = RightNeighbor(id);

    if (g_eating[left])
        return 0;

    if (g_eating[right])
        return 0;

    return 1;
}
```

```
void AddRequest(int id)
{
    if (g_waiting[id]) // Если философ уже ждёт, ничего не
делаем
        return;

    g_request_queue[g_queue_tail] = id; // Иначе кладём его айди
в очередь
    g_queue_tail = (g_queue_tail + 1) % PHIL_COUNT;
    g_queue_count++;

    g_waiting[id] = 1; // Философ ждёт
}

int PopRequest(int* id) // Функция, к-ая достаёт запрос из
очереди
{
    if (g_queue_count == 0) // Если очередь пустая, возвращаем 0
        return 0;

    *id = g_request_queue[g_queue_head]; // Иначе берём первого
философа из очереди
    g_queue_head = (g_queue_head + 1) % PHIL_COUNT; // Сдвигаем
head
    g_queue_count--; // Уменьшаем count;

    return 1;
}

void TryGivePermissions() // Главная функция управляющего
{
    /*  Функция смотрит очередь запросов и разрешает есть
        тем философам, у кого соседи сейчас не едят
    */
    int checked = g_queue_count; /*  Запоминаем
```

```
     сколько запросов было в очереди на момент начала проверки
*/
    /* checked нужен, чтобы за один проход проверить только
текущую очередь
    и не зациклиться на философах, которым сейчас нельзя есть.
    Если им нельзя, они возвращаются в очередь и будут проверены
позже,
    когда изменится ситуация. */
    for (int i = 0; i < checked; i++)
    {
        int id; // Философ

        if (!PopRequest(&id)) // Берём 1-ого философа из очереди
            break;

        if (!g_finished && CanEat(id)) // Если программа ещё не
заканчивается и философ может есть
        {
            g_waiting[id] = 0;
            g_eating[id] = 1;
            sem_post(&g_permission_sem[id]); // Разрешаем
            // sem_post будит именно этого философа
        }
        else // Если есть нельзя, кладём его обратно в конец
очереди
        {
            g_request_queue[g_queue_tail] = id;
            g_queue_tail = (g_queue_tail + 1) % PHIL_COUNT;
            g_queue_count++;
        }
    }
}

// pthread_create требует, чтобы функция имела такой вид
void* PhilosopherThread(void* param) // Функция Философа
{
```

```
    int id = (int)((long)param); // Извлекаем id философа

    while (1)
    {
        SleepMs(g_phil_time); //Философ думает

        if (GetTimeMs() >= g_total_time) // Если время вышло,
есть не просит
            break;


        /* Т.к дальше проверяется общий флаг завершения и
происходит
           добавление в общую очередь запросов, и эту общую
очередь запросов
           может использовать несколько потоков, следовательно,
доступ
           к этим данным должен быть защищён
        */
        pthread_mutex_lock(&g_mutex);

        if (!g_finished)
        {
            AddRequest(id); // Хочет кушать, философ добавляет
себя в очередь
            sem_post(&g_manager_sem); // Обращается к
управляющеу потоку
        }

        pthread_mutex_unlock(&g_mutex);

        sem_wait(&g_permission_sem[id]); // Философ ждёт
разрешения есть

        pthread_mutex_lock(&g_mutex);
```

```
        // Проверка на завершение программы
        if (g_finished && !g_eating[id])
        {
            pthread_mutex_unlock(&g_mutex);
            break;
        }

        // Философ начал есть
        PrintState(id, STATE_THINKING, STATE_EATING);

        pthread_mutex_unlock(&g_mutex);

        SleepMs(g_phil_time); // Ест PHIL time

        pthread_mutex_lock(&g_mutex);

        PrintState(id, STATE_EATING, STATE_THINKING);

        g_eating[id] = 0;
        g_state[id] = STATE_THINKING;

        sem_post(&g_manager_sem); // увеличиваем семафор на 1

        pthread_mutex_unlock(&g_mutex);
    }

    pthread_mutex_lock(&g_mutex);

    g_active_philosophers--;
    sem_post(&g_manager_sem);

    pthread_mutex_unlock(&g_mutex);

    return 0;
}
```

```
void* ManagerThread(void* param)
{
    while (1)
    {
        /* В начале управляющий поток спит, т.к мы
           проинициализировали его семафор нулём.
           Он проснётся тогда, когда философ увеличит
           значение семафора упраляющего через sem_post,
           т.е обратится к нему. До этого ему нечего делать
        */
        sem_wait(&g_manager_sem);

        /* Дальше управляющий поток работает с общими данными,
            т.к дальше он будет читать/менять общие данные,
связанные
            с очередью, к-ые могут изменить потоки-философы. */
        pthread_mutex_lock(&g_mutex);

        TryGivePermissions(); // Пробуем выдать разрешения есть
философами

        if (g_finished && g_active_philosophers == 0)
        {
            pthread_mutex_unlock(&g_mutex);
            break;
        }

        pthread_mutex_unlock(&g_mutex);
    }

    return 0;
}

int main(int argc, char* argv[]) // ./phil 5000 100
{
    if (argc != 3) // Проверяем, что аргументов ровно 3
```

```
    {
        return 1;
    }

    g_total_time = atoi(argv[1]); // Переводим строки в числа
    g_phil_time = atoi(argv[2]);

    if (g_total_time <= 0 || g_phil_time <= 0)
    {
        return 1;
    }

    // Запоминаем время старта программы
    clock_gettime(CLOCK_REALTIME, &g_start_time);

    pthread_mutex_init(&g_mutex, 0); // Инициализируем мьютекс

    sem_init(&g_manager_sem, 0, 0); // Семафор управляющего
потока
    // Начальное значение – 0

    // Для каждого философа создаём личный семафор разрешения
    for (int i = 0; i < PHIL_COUNT; i++)
    {
        // Начальное значение 0 – философы будут ждать пока
управляющий им не разрешит
        sem_init(&g_permission_sem[i], 0, 0);

        g_state[i] = STATE_THINKING; // В начале все думают
        g_waiting[i] = 0; // никто ещё не ждёт разрешения
        g_eating[i] = 0;  // и никто не ест
        g_eat_count[i] = 0;
        g_think_count[i] = 0;
    }

    // Создаём управляющий поток
```

```
    // Он будет ждать запросы философов и выдавать разрешения
    pthread_create(&g_manager, 0, ManagerThread, 0);

    // Создаём 5 потоков философов
    for (int i = 0; i < PHIL_COUNT; i++)
    {
        pthread_create(&g_philosophers[i], 0, PhilosopherThread,
(void*)((long)i));
    }

    SleepMs(g_total_time);

    pthread_mutex_lock(&g_mutex);

    g_finished = 1;

    // Надо разбудить философов, к-ые ожидают разрешения поесть
    for (int i = 0; i < PHIL_COUNT; i++)
    {
        sem_post(&g_permission_sem[i]);
    }

    sem_post(&g_manager_sem); // Программа начинает завершаться
    // Надо разбудить управляющего

    pthread_mutex_unlock(&g_mutex);

    // Нужно подождать всех философов, к-ые могут есть
    for (int i = 0; i < PHIL_COUNT; i++)
    {
        pthread_join(g_philosophers[i], 0); // ждём когда
указанный поток завершится
    }

    sem_post(&g_manager_sem);
    pthread_join(g_manager, 0);
```

```
    FILE* stats = fopen("stats.txt", "w");
    if (stats != NULL)
    {
        fprintf(stats, "PHILOSOPHER E T\n");

        for (int i = 0; i < PHIL_COUNT; i++)
        {
            fprintf(stats, "%d %d %d\n", i + 1, g_eat_count[i],
g_think_count[i]);
        }

        fclose(stats);
    }

    for (int i = 0; i < PHIL_COUNT; i++)
    {
        sem_destroy(&g_permission_sem[i]);
    }

    sem_destroy(&g_manager_sem);
    pthread_mutex_destroy(&g_mutex);

    return 0;
}
```

Приложение Д «chess.cpp»

```
#include <windows.h>
#include <stdio.h>

#define MAX_THREADS 1024

struct ChessTask // Одна задача для потока
{
    int start_pos; // С какой клетки начинаем перебор
    int left_to_place; // Сколько королей ещё надо поставить
    unsigned char* board; // копия текущей доски для этой задачи
};

int g_threads_count = 0; // Кол-во потоков
int g_n = 0; // Размер доски
int g_l = 0; // Сколько новых королей надо поставить
int g_k = 0; // Сколько королей уже стоит

unsigned char* g_initial_board = NULL; // Начальная доска

ChessTask* g_tasks = NULL; // Массив задач
int g_task_count = 0;      // Сколько задач всего создано
int g_next_task = 0;       // Какую задачу следующий поток
должен взять

unsigned long long g_answer = 0; // Общее кол-во решений

HANDLE g_start_event = NULL;  // Событие старт, чтобы потоки не
начали считать до таймера
HANDLE g_task_mutex = NULL;   // Мьютекс для выдачи задач
HANDLE g_answer_mutex = NULL; // Мьютекс для общего ответа

int GetIndex(int x, int y) // Переводим координаты клетки в
индекс одномерного массива
{
    return x * g_n + y;
}

int IsInside(int x, int y) // Проверка, что координаты внутри
доски
{
    return x >= 0 && x < g_n && y >= 0 && y < g_n;
}

unsigned char* CopyBoard(unsigned char* board) // создаём копию
доски для каждой задачи
{
    int size = g_n * g_n;
    unsigned char* copy = new unsigned char[size];

    for (int i = 0; i < size; i++)
    {
```

```
        copy[i] = board[i];
    }

    return copy;
}

int CanPlaceKing(unsigned char* board, int pos)
{
    if (board[pos]) // Если клетка уже занята, ставить нельзя
        return 0;

    // Переводим pos в координаты
    int x = pos / g_n;
    int y = pos % g_n;

    for (int dx = -1; dx <= 1; dx++)
    {
        for (int dy = -1; dy <= 1; dy++)
        {
            if (dx == 0 && dy == 0) // Пропускаем саму клетку
                continue;

            int nx = x + dx;
            int ny = y + dy;

            if (IsInside(nx, ny)) // Если сосед внутри доски
            {
                int index = GetIndex(nx, ny);

                if (board[index]) // Проверяем есть ли король
                    return 0;
            }
        }
    }

    return 1; // Ставить можно
}

// Сколько способов поставить left_to_place королей, начиная с
клетки start_pos.
unsigned long long CountWays(unsigned char* board, int
start_pos, int left_to_place)
{
    // Если ставить больше ничего не надо, то найдено одно
решение
    if (left_to_place == 0)
        return 1;

    int size = g_n * g_n;

    // Если клеток осталось меньше, чем королей, то решений нет
    if (size - start_pos < left_to_place)
        return 0;
```

```
    unsigned long long count = 0;

    // Пробуем поставить следующего короля в каждую клетку от
start_pos до конца
    for (int pos = start_pos; pos < size; pos++)
    {
        if (CanPlaceKing(board, pos)) // Если клетка безопасная
        {
            board[pos] = 1; // ставим короля

            count += CountWays(board, pos + 1, left_to_place -
1);
            /* Рекурсия пробует поставить очередного короля в
каждую безопасную клетку.
            После постановки она вызывает себя, чтобы поставить
оставшихся королей дальше по доске.
            Когда ставить больше не надо, возвращает 1 найденное
решение.
            Потом король убирается, чтобы попробовать другие
варианты. */
            board[pos] = 0;
        }
    }

    return count;
}

/* GetNextTask() под мьютексом выдаёт потоку следующую задачу из
массива g_tasks.
 Если задачи закончились, возвращает 0. Мьютекс нужен, чтобы два
потока не получили
 одну и ту же задачу. */
int GetNextTask(ChessTask* task) // Взять следующую задачу и
добавить результат
{
    int has_task = 0; // считаем, что задачи нет

    // Захватываем мьютекс очереди задач
    WaitForSingleObject(g_task_mutex, INFINITE);

    // Проверяем, остались ли задачи
    if (g_next_task < g_task_count)
    {
        task->start_pos = g_tasks[g_next_task].start_pos;
        task->left_to_place =
g_tasks[g_next_task].left_to_place;
        task->board = g_tasks[g_next_task].board;

        g_next_task++;
        has_task = 1;
    }
```

```
    ReleaseMutex(g_task_mutex);

    return has_task;
}

// Прибавляем локальный результат потока к общему ответу
void AddAnswer(unsigned long long value)
{
    WaitForSingleObject(g_answer_mutex, INFINITE);

    g_answer += value;

    ReleaseMutex(g_answer_mutex);
}

DWORD WINAPI WorkerThread(void* param)
{
    /* Локальный счётчик нужен, чтобы поток считал
    свои решения без постоянного обращения к общему
    g_answer. В общий ответ он добавляет результат
    один раз в конце, под мьютексом. */
    unsigned long long local_answer = 0;

    WaitForSingleObject(g_start_event, INFINITE);

    while (1)
    {
        ChessTask task;

        if (!GetNextTask(&task))
            break;

        local_answer += CountWays(task.board, task.start_pos,
task.left_to_place);
    }

    AddAnswer(local_answer);

    return 0;
}

void CreateTasks()
{
    int size = g_n * g_n;

    if (g_l == 0) // Если новых королей ставить не надо, то уже
есть одно решение: начальная доска
    {
        g_answer = 1;
        return;
    }

    g_tasks = new ChessTask[size]; // Создаём массив задач
```

```
    g_task_count = 0;
    g_next_task = 0;

    // Пробуем поставить первого нового короля в каждую клетку
    for (int pos = 0; pos < size; pos++)
    {
        // Если клетка безопасна
        if (CanPlaceKing(g_initial_board, pos))
        {
            // Создаём копию начальной доски
            unsigned char* board = CopyBoard(g_initial_board);
            board[pos] = 1; // ставим на эту клетку короля

            // Создаём задачу
            g_tasks[g_task_count].start_pos = pos + 1;
            g_tasks[g_task_count].left_to_place = g_l - 1;
            g_tasks[g_task_count].board = board;

            g_task_count++;
        }
    }
}

int main()
{
    FILE* input = fopen("input.txt", "r");
    if (input == NULL)
    {
        return 1;
    }

    fscanf(input, "%d", &g_threads_count);
    fscanf(input, "%d %d %d", &g_n, &g_l, &g_k);

    if (g_threads_count <= 0 || g_threads_count > MAX_THREADS ||
g_n <= 0 || g_l < 0 || g_k < 0)
    {
        fclose(input);
        return 1;
    }

    int size = g_n * g_n;

    g_initial_board = new unsigned char[size];

    for (int i = 0; i < size; i++)
    {
        g_initial_board[i] = 0;
    }

    for (int i = 0; i < g_k; i++)
    {
        int x, y;
```

```
            fscanf(input, "%d %d", &x, &y);

            if (x >= 0 && x < g_n && y >= 0 && y < g_n)
            {
                g_initial_board[GetIndex(x, y)] = 1;
            }
        }

        fclose(input);

        g_start_event = CreateEvent(NULL, TRUE, FALSE, NULL);
        g_task_mutex = CreateMutex(NULL, FALSE, NULL);
        g_answer_mutex = CreateMutex(NULL, FALSE, NULL);

        if (g_start_event == NULL || g_task_mutex == NULL ||
g_answer_mutex == NULL)
        {
            return 1;
        }

        CreateTasks();

        HANDLE* threads = new HANDLE[g_threads_count];

        for (int i = 0; i < g_threads_count; i++)
        {
            threads[i] = CreateThread(NULL, 0, WorkerThread, NULL,
0, NULL);

            if (threads[i] == NULL)
            {
                return 1;
            }
        }

        DWORD startTime = GetTickCount();

        SetEvent(g_start_event);

        for (int i = 0; i < g_threads_count; i++)
        {
            WaitForSingleObject(threads[i], INFINITE);
        }

        DWORD finishTime = GetTickCount();
        DWORD workTime = finishTime - startTime;

        FILE* output = fopen("output.txt", "w");
        if (output != NULL)
        {
            fprintf(output, "%I64u\n", g_answer);
            fclose(output);
        }
```

```
    FILE* timeFile = fopen("time.txt", "w");
    if (timeFile != NULL)
    {
        fprintf(timeFile, "%lu\n", (unsigned long)workTime);
        fclose(timeFile);
    }

    for (int i = 0; i < g_threads_count; i++)
    {
        CloseHandle(threads[i]);
    }

    CloseHandle(g_start_event);
    CloseHandle(g_task_mutex);
    CloseHandle(g_answer_mutex);

    if (g_tasks != NULL)
    {
        for (int i = 0; i < g_task_count; i++)
        {
            delete[] g_tasks[i].board;
        }

        delete[] g_tasks;
    }

    delete[] threads;
    delete[] g_initial_board;

    return 0;
}
```